

SECOND EDITION

THE

PS2

LESSONS
LEARNED
(SO FAR)

PROGRAMMING
LANGUAGE

NINJA DYNAMICS
JUNE 2026

Contents

Chapter 1 - The PS2 Mental Model

Chapter 2 - Source Boundaries and Port Architecture

Chapter 3 - Boot, IOP Bring-Up, and Standalone Reality

Chapter 4 - Toolchain and Build-System Traps

Chapter 5 - GS Display, Framebuffers, and Video Modes

Chapter 6 - Aspect Ratio and Full-Frame Coverage

Chapter 7 - ps2gl State, Batching, and Frame Boundaries

Chapter 8 - Depth, Blending, and 2D Primitives

Chapter 9 - Geometry, Clipping, Projection, and Display Lists

Chapter 10 - Texture Formats, Alpha, and Filtering

Chapter 11 - Manual Mipmaps, Dither, and GS Texture Precision

Chapter 12 - GS and EE Memory Management

Chapter 13 - Effects and Scene-Specific Rendering

Chapter 14 - Input, Debug UI, and Save Data

Chapter 15 - SPU2 SFX Fundamentals

Chapter 16 - Music Streaming and Custom IOP Modules

Chapter 17 - Validation Doctrine

Chapter 18 - ps2gl and ps2stuff Fork Delta

Chapter 19 - GS Hardware Fog: The Unit Works, the Fork Never Wired It

Chapter 20 - Fork Stability, Per-Vertex Alpha, and Batched Submission

Chapter 21 - Coexisting Mipmaps and the Long-Run Reset

Chapter 22 - Writing VU1 Microcode Through VCL: The Clip Renderer

Chapter 23 - Measuring the Wall, and Kicking Twice

Chapter 24 - The Heap Smasher: Forensics When the Crash Site Lies

Chapter 25 - The Sixty Lock: One Kick, One Page, and the Flat Matrix

Appendix A - Quick Rules

Appendix B - Case Study: HyperSolar PS2 Port in One Page

The PS2 Programming Language

Lessons Learned (So Far)

by Ninja Dynamics

Public Release Edition, June 2026

This handbook is distilled from HyperSolar's PS2 work. It is written for the person who already has the SDK, already has the manuals, and is now staring at a black screen, a silent SPU2, a wrong-colored CLUT, or a render path that works in PCSX2 and fails on hardware.

It is not a replacement for Sony's manuals. It is the scar tissue layer: the rules that only became obvious after ps2gl, the GS, SPU2, SIF RPC, PCSX2, ps2link, and real hardware disagreed with one another.

The promise is practical: every chapter should leave you with a rule you can use in a real game, a failure mode you can recognize, and a reference you can open when you need to go deeper.

Public release note: this edition is written to stand alone. Implementation names, helper names, and subsystem names appear as concrete examples, not as dependencies the reader must have.

Why does this exist

Most of what is hard about the PS2 is not in any one manual. The Sony manuals are excellent first-principles references — they tell you exactly what every GS register does and exactly how an SPU2 voice is structured. What they cannot tell you is which of those facts will quietly ruin your week: that a 16-bit color buffer paired with a 24-bit Z buffer renders perfectly in an emulator and corrupts on hardware; that `pglFinish()` destroys your GL context instead of flushing it; that a buffer-loaded IRX can report success and never register its RPC server. Those are emergent properties of the *combination* — EE plus GS plus IOP plus SPU2 plus a forked OpenGL-ish library plus an emulator plus a network loader — and they only show up when the pieces disagree.

This handbook is the record of those disagreements and how they were resolved. It is deliberately opinionated: where a rule has survived real hardware validation it is stated plainly, and where a fix is empirical-but-unproven it says so.

How to read this handbook

Each chapter is self-contained and built around the same spine: a rule you can apply, the failure mode that makes you reach for it, and a pointer to the Sony manual chapter that explains the underlying hardware. Read straight through for a tour of the whole port, or jump to the chapter that matches your current black screen. Chapter 1 is the mental model everything else assumes; Appendix A is the whole book compressed to a checklist; Appendix B is the port told as a timeline.

Conventions

- **"Validate on hardware"** appears often and is not boilerplate. PCSX2 is a different witness from a real console (see Chapter 17). When a rule is hardware-only, the text says which class of

bug the emulator hides.

- **Manual citations** name the Sony manual and the relevant chapter so you know where to open for first-principles detail. The GS User's Manual splits into *Local Memory* (Ch 2, pixel-storage modes and page geometry), *Drawing Function* (Ch 3, texture mapping, alpha-blending, dithering, pixel test), *Image Data Transmission* (Ch 4, EE↔GS transfers and GIF), *CRTC* (Ch 5, display/video output), and *Registers* (Ch 7) — those splits are referenced by number throughout.
- **Confidence is marked.** Proven rules are stated as rules; inferred ones are flagged as empirical so you do not inherit a guess as a fact.

Sources and reference shelf:

- HyperSolar PS2 development notes, distilled and rewritten for public release.
- Ninja Dynamics, ps2gl and ps2stuff Fork Delta Report, June 2026.
- Sony PlayStation 2 GS User's Manual: GS registers, display environment, pixel storage modes, CLUTs, texture registers, alpha blending, scissor, GIF packet formats, and image transfers.
- Sony PlayStation 2 GS User's Manual Supplement: GS errata and corner cases.
- Sony PlayStation 2 EE Overview Manual: EE, VU0/VU1, GIF, VIF, DMAC, IPU, and the system architecture.
- Sony PlayStation 2 EE User's Manual: DMAC, GIF, VIF, timers, INTC, scratchpad, and transfer behavior.
- Sony PlayStation 2 EE Core User's Manual: R5900 CPU, caches, MMU, COP0, and pipeline behavior.
- Sony PlayStation 2 EE Core Instruction Set Manual: R5900 and MMI instructions.
- Sony PlayStation 2 VU User's Manual and VU Instruction Manual: VU architecture, micro-mode, and microinstructions.
- Sony PlayStation 2 SPU2 Overview Manual: two cores, 24 ADPCM voices per core, 2 MB SPU2 RAM, voice registers, PITCH, ADSR, KON/KOFF, VMIX, and mixing.
- Community PS2 texture-format references are useful quick companions to the GS manual's PSM and CLUT tables.

Manual citations in this handbook name the relevant Sony manual by title. No quoted manual text is required to use this handbook; the manual references are there so you know which hardware chapter to open when you want first-principles detail.

Chapter 1 - The PS2 Mental Model

Relevant manuals: EE Overview Manual Ch 2 (Architecture Overview) and Ch 3 (Functional Overview) for how the EE core, VU0/VU1, GIF, VIF, DMAC, and IPU connect and feed the GS; GS User's Manual Ch 1 (Overview) for the GS block configuration; SPU2 Overview Manual Ch 1 (Overview of SPU2) for the audio side.

Before any specific rule, internalize the shape of the machine — almost every trap later in the handbook is really a consequence of getting one of these seams wrong (an EE pointer handed to the IOP, a stream forced through a DMA boundary, an emulator standing in for a CPU it does not fully model).

The PS2 is several machines tied together by DMA and RPC-like boundaries. Treat those boundaries as real architectural seams.

The EE runs the game, simulation, asset orchestration, and most rendering glue. The VUs and ps2gl handle transform/render submission for supported paths, but the case-study renderer also uses an EE CPU software clipper for geometry that ps2gl cannot safely clip. The GS is not a modern forgiving GPU; it is a rasterizer with strict VRAM page geometry, fixed-point coordinate limits, explicit texture registers, and very sticky state.

The IOP owns many services that look "system level" from the EE: modules, filesystems, memory cards, libsd/ps2snd, USB mass storage, and custom IRX code. The EE speaks to it through SIF RPC and DMA. If an API takes a pointer and the work happens on the IOP, ask which CPU's address space the pointer belongs to.

The SPU2 is a local-memory ADPCM synth, not a stream-player. It gives excellent per-voice pitch/volume control once samples are resident in SPU2 RAM. Long music requires streaming into SPU2 memory while the voice plays.

PC, PCSX2, ps2link, and real hardware are four different witnesses. PCSX2 is great for correctness and crashes, but it can hide GS display-register bugs, texture precision artifacts, mip blend quantization, and some hardware-only filtering behavior. ps2link can mask standalone IOP boot problems because it leaves the IOP in a friendlier state than a disc, USB boot, or emulator launch. Real hardware remains the final judge.

Chapter 2 - Source Boundaries and Port Architecture

Relevant manuals: mostly architectural — EE Overview Manual Ch 1 (Architecture Policy) frames why the EE/GS/IOP division of labor exists in the first place. This chapter is otherwise engineering discipline drawn from the case study, not hardware behavior.

This chapter is about a different kind of correctness than the rest of the handbook: not "what does the hardware do" but "where does each responsibility live in your code." It earns its place because every PS2-specific bug is cheaper to fix when the platform code is a thin, parameter-fed library than when it has quietly grown into a second copy of the game with its own drifting scene order.

The PS2 backend should be a library, not a parallel game. In the case-study codebase that backend is named `playstation2.c`; in another project it may have another name. The important shape is a hardware/service layer that shared game and render code call into.

It may own:

- pad adapter code;
- embedded asset registry callbacks;
- GS-safe texture helpers;
- alpha conversion and blend-state helpers;
- the software clipper;
- display-list compile/call helpers;
- PS2 draw primitives such as skybox, floor, haze, particles, reticles, sun, speedlines, and lens flare;
- scene resource installers that turn already-loaded assets into PS2 helper meshes, display lists, or baked textures;
- memory-card blob I/O and low-level PS2 service code.

It should not:

- read `game.*`, except for the explicit pad-write adapter pattern;
- name game asset files directly;
- compose scenes;
- own stage sequencing, gameplay state transitions, timers, scoring, or sim behavior.

Every value derived from game state crosses into the PS2 helper as a parameter: camera, anchor, floor type, scroll values, model transforms, lighting, tint, bounding sphere, sun position, or effect intensity. This keeps the cross-platform files as the source of truth.

The scene graph should remain shared. In the case study, duplicated PS2 scene code drifted in draw order. That drift was not cosmetic: haze drew after particles and made explosions look behind fog. Keep new scene elements in the shared scene function and put PS2-specific behavior at leaf helpers.

When porting a cross-platform render function, grep the original for `game.stage->` and `game.*` reads. Each read is either a parameter the PS2 dispatch must forward or a sign that the helper boundary is wrong. The lava stage exposed this: a PS2 floor helper hardcoded the water floor composition and the lava stage became double-layered and transparent.

Stage-shaped resources are now real per-stage resources on PS2. Skyboxes and floors load from the stage table, call the CSV texture pipeline, and install the PS2 form. Only the active scene's GS resources should be resident. Mipped floors need a deep unload because their locked mip levels and global MIPTBP ownership outlive a plain texture unload.

The backend API has two classes of calls:

- public one-shot draw calls and public `*_begin()` calls must fence their own GS state;
- hot-loop `*_add()` or `*_draw()` calls between begin/end should be lean and inherit state from the matching begin.

The F22 inside-out bug is the warning label. Reticles restored generic raylib state that re-enabled culling. If `ps2_render_player()` trusts prior state, the reversed GLB winding gets culled from the visible side.

Chapter 3 - Boot, IOP Bring-Up, and Standalone Reality

Relevant manuals: EE Overview Manual Ch 2 (Architecture Overview) for the EE/IOP/SIF relationship; EE User's Manual for the DMAC and SIF transfer paths the RPC layer rides on. For audio-side consequences, see SPU2 Overview Manual Ch 1.

The IOP is the part of the machine most likely to lie to you at boot. It will accept a module, hand you back a positive id, and quietly never finish bringing it up — and the one tool you would normally use to see this (`printf`) is gone the moment you leave the network loader. This chapter is the bring-up ritual and the breadcrumb techniques that make a silent IOP visible.

Do standalone IOP bring-up before loading buffer IRX modules. A module loaded with `SifExecModuleBuffer` can return a positive module id and still never register its RPC server if the IOP is in the wrong inherited state. `ps2link` masks this because its loader leaves the IOP usable; PCSX2 and real disc/USB standalone boots do not.

The reliable sequence is:

```
SifInitRpc(0);
SifIopReset("", 0);
while (!SifIopSync()) { }
SifInitRpc(0);
sbv_patch_enable_lmb();
sbv_patch_disable_prefix_check();
```

Link `-lpatches` and include the IOP control and SBV patch headers. This belongs at the top of pre-init, before memory-card modules, `libsd/ps2snd`, BDM, custom IRX modules, or any other buffer-loaded code.

`SifIopReset` drops `ps2link` console and `host: fileio`. That is acceptable only if the game no longer needs `host:` after boot. Once assets are embedded and USB or disc paths are explicit, this makes the dev run behave like the shipped standalone run, which is exactly what caught the silent-audio bug.

Do not equate "works through `ps2link` on hardware" with "works standalone." The audio stack proved the danger: `ps2snd.irx` worked over `ps2link`, but in standalone boot its `BINDID_PS2SND` RPC never came up until the IOP reset and SBV patches were added.

Make the reset build-conditional so it does not cost you the dev loop. `SifIopReset` reboots the whole IOP, and under network boot `ps2link`'s ethernet driver (SMAP) lives there — so the reset kills `ps2link`'s network. The EE program keeps running, but `ps2client reset` / `printf` die and you must physically power-cycle to reload `ps2link` before the next deploy (telltale: `ping` answers right after a hard reset, then goes unreachable the instant the ELF runs). The reset is only needed standalone — under `ps2link` the IOP is already primed for buffer-loaded IRX — so guard it with a build flag: `#ifndef PS2_PS2LINK` around the reset / sync / re-init, defined only for the `ps2client` deploy goal, keeping `SifInitRpc(0)` and the SBV patches on both paths. The standalone build stays self-contained; the network-boot dev build keeps `ps2link` alive so reset/printf survive and iteration needs no power-cycle.

When debugging standalone or PCSX2 boot hangs, `printf` is gone. `ps2sdk printf` routes to `ps2link`, and PCSX2's trace log has hardware trace categories, not a program-console stream. Use GS color breadcrumbs:

```
BeginDrawing();  
ClearBackground(SOME_COLOR);  
EndDrawing();  
ps2_frame_end();
```

The screen freezes on the last completed color. PCSX2 logs can still distinguish exceptions from spins. If there is no EE exception but the log shows timeout loop skipping, suspect a bind loop or wait.

`nopdelay()` is expensive. It is about one million nops, roughly 17 ms per call. Big SDK-style loops survive only because they break on the first success. If a failure path can run to completion, a `100000 * nopdelay()` wait is effectively a hang. Use small bounded counts.

`sceSifBindRpc(cd, sid, 0)` blocks until the IOP replies. An unregistered server does not reply. For probes, use `SIF_RPC_M_NOWAIT`, poll `cd->server`, and re-send the bind occasionally because an unregistered SID can be silently dropped before the server appears.

Chapter 4 - Toolchain and Build-System Traps

Relevant manuals: EE Core User's Manual and EE Core Instruction Set Manual cover the R5900 the toolchain targets, but most of this chapter is GCC/binutils/ps2sdk behavior, not Sony hardware behavior.

These are the bugs that never reach the hardware because they stop at the compiler or linker — yet they cost real days, because the error messages point nowhere near the cause (a predefined macro turning a type into a number, a native-Windows assembler rejecting a POSIX path, a missing inline definition surfacing as a link error on one platform only).

The DreamSDK PS2 assembler is a native Windows executable. When MSYS invokes `mips64r5900el-ps2-elf-as.exe`, `.incbin "/c/Users/..."` fails because that path is not a Win32 path. Generate mixed Windows paths for `.incbin`, for example via `cygpath -m`, with a fallback for non-MSYS environments.

The EE compiler predefines `mips` as a macro. Do not name variables, fields, or parameters `mips`. It preprocesses to `1`, creating baffling syntax errors such as `CMMTexture* 1[6]`. Use `levels`, `mip_list`, or another name.

`RAYMATH_IMPLEMENTATION` must exist in exactly one translation unit for PS2. The PS2 toolchain plus `raylib4ps2` can fail to inline functions that other platforms resolve elsewhere, leading to undefined references such as `MatrixMultiply`. Put the implementation define in exactly one PS2-only backend translation unit.

When using forked libraries, link by explicit path and include their headers ahead of toolchain headers. In a sibling-fork layout, build `ps2stuff` as a local archive, link that archive by path, and do not install it over the toolchain copy. `ps2gl` must also compile against the forked `ps2stuff` headers if it uses fork-only APIs such as GS memory info.

Canary banners are a cheap sanity check for fork linkage. Make them print once at startup from `raylib`, `ps2gl`, and `ps2stuff`. A build-script stamp only rewrites an existing string; it cannot insert a missing `printf`.

Chapter 5 - GS Display, Framebuffers, and Video Modes

Relevant manuals: GS User's Manual Ch 5 (CRTC) for the DISPLAY/PMODE registers, magnification, and video output; Ch 2 (Local Memory) for page geometry and framebuffer storage; Ch 7 (Registers) for BGCOLOR and the privileged register addresses; GS User's Manual Supplement for errata.

This is the chapter where the emulator lies most. The GS display path — sync timing, the DISPLAY register's magnification, framebuffer page alignment, the color/Z pairing, BGCOLOR — is largely normalized away by PCSX2 and only tells the truth on a real console with a real TV. Treat every rule here as hardware-validated, because each one was caught only after PCSX2 said it was fine.

Do not call libgraph's `graph_set_mode` or `graph_set_screen` on top of ps2gl. libgraph and ps2gl both program the GS display read-circuit registers. Calling libgraph corrupts ps2gl's display environment and can crash even in the default mode. Use `SetGsCrt` for sync-generator changes and ps2gl/fork helpers for the display registers that ps2gl owns.

`SetGsCrt` is only half of a video-mode switch. It changes scan timing, not the GS DISPLAY register's position, magnification, and size. ps2gl re-sends its display environment each frame, so a mode switch must update ps2gl's live display environment too.

Use the corrected ps2sdk constant: `GS_MODE_DTV_480P == 0x50`. An older lesson recorded `0x32`; that is superseded by the later PS2 video-mode work.

The case-study progressive layout is 640x448p, not 480p. The reasons are pure GS layout math:

- 448 is exactly 7 rows of 64-line 16-bit GS pages.
- 480 is 7.5 rows and needs padding to 512 lines in memory.
- 448p can share the same visible `SCREEN_W/H` and texture slot map as 448i.
- A 640x448 16-bit framebuffer uses 70 pages, matching the stock 448i footprint.

GS framebuffers must be page-row aligned. A GS page is 64x32 for 32/24-bit formats and 64x64 for 16-bit formats. Do not size framebuffers by `width * height * bpp / 8192` unless the height is page aligned. For a 16-bit 640x480 buffer, 480 lines address into 7.5 page rows; the bottom-right portion spills into the next buffer unless the reservation is rounded to 512 lines.

Color and Z formats must share page geometry. Pair 16-bit color with 16-bit Z, and pair 32-bit color with 24-bit Z. A 16-bit color buffer with 24-bit Z may look fine in PCSX2 but corrupts on real GS hardware because color and Z pages advance on different row heights.

The cost of 16-bit Z is coarser depth. Thin double-sided geometry can z-fight. Fix that with culling, not by switching to 24-bit Z. The case-study F22 GLB has reversed winding relative to ps2gl's `GL_BACK` convention, so the player model needs `glCullFace(GL_FRONT)`.

For DTV progressive, `DISPLAY.magH` must match the faster scan clock. Interlaced NTSC/PAL used `magH = 4`; DTV progressive needs `magH = 2`. Leaving the old value makes the image twice as wide on hardware. PCSX2 can normalize this away, so validate display-register work on a real console.

Runtime GS layout switching is possible, but the safe ritual matters:

- `pglFinish()` is not a flush; it destroys the GL context. Drain with `pglWaitForVU1()` and `pglWaitForVSync()`.
- Remove and rebuild GS memory slots, including locked framebuffer slots.
- Re-add framebuffer and Z slots, bind areas, set draw buffers, call `SetGsCrt`, set display buffers, then set the ps2gl video mode.
- Most textures re-upload lazily after the slot wipe, but mipmapped textures bake GS addresses into MIPTBP. Re-create them after the switch.
- Black-bracket the transition so uninitialized buffers and slot rebuilds do not scan out as VRAM garbage.
- The remaining blink is GS scan-generator resync; it is hardware behavior.

The GS `BGCOLOR` register controls the margins outside the active framebuffer. `ClearBackground` does not touch it. Soft reset does not reliably clear it. Write it as a 64-bit store to physical `0x120000E0`, not a `kseg1` alias. Re-zero it after `EndDrawing()` and after video-mode application. Also re-zero it before known long mid-frame stalls such as scene loads; otherwise a previously stomped color can show in the side bars for the duration of the stall.

The interlace flicker filter is the two-read-circuit merge, not a post pass and not `S.MODE2.FFMD`. The GS has two display read circuits (`P.MODE.EN1 / EN2`). Enable both on the *same* framebuffer, offset read circuit 1 down one scanline (`DISPFB1.DBY = 1`), and const-alpha blend them (`P.MODE.MMOD=1`, `ALP`, `SLBG=0`): a 2-tap vertical low-pass on scanout that settles interlaced field twitter (thin-line flicker, distant-edge shimmer) at zero fill-rate cost. It is the classic PS2 softening most interlaced-era titles shipped. Three facts make it work rather than ghost:

- It is meaningful only in interlaced modes. Progressive output has no field twitter, so the blend only softens the image — gate it off there.
- Both read circuits must be repointed on every buffer flip. The usual swap path updates only `DISPFB2`'s base address; with the filter on, RC1 must be repointed to the same buffer too, or the two circuits blend two different frames and the image ghosts. The one-line `DBY` offset lives in the register and survives an address-only update.
- A live raster re-center (the `DISPLAY`-only, no-border-flash path) must move `DISPLAY1` alongside `DISPLAY2`, or RC1 and RC2 pan apart into a misaligned double image.

It is a whole-scanout effect, so it softens the HUD too; that cannot be excluded without a separate render-to-texture pass. Validate the blend alpha on hardware: PCSX2 does not reproduce CRT interlace twitter, so it is a poor witness for whether a given strength helps, and RC1's `+1` line reads one scanline past the buffer on the last row.

Chapter 6 - Aspect Ratio and Full-Frame Coverage

Relevant manuals: GS User's Manual Ch 5 (CRTC) for the display environment — how the read circuit magnifies and positions the framebuffer onto the output raster, which is the root cause of the stretch this chapter corrects.

A point that surprises most newcomers: a plain 4:3 PS2 framebuffer is *not* geometrically neutral. The display hardware stretches the stored lines to fill the raster, so "render 640×448 and forget about it" ships a subtly squished world. This chapter is the correction math and — just as important — the separation between *aspect* (a projection scale) and *placement* (a HUD re-anchor), which are easy to conflate and painful to debug once you have.

The 640x448 PS2 framebuffer is not geometrically neutral on a 4:3 display. The display stretches 448 lines to the TV raster height, making the scene vertically stretched relative to DC and PC. A 640x448 renderer can correct this by applying a centered horizontal scale:

```
xscale = (float)NATIVE_H / (float)SCREEN_H; /* 480 / 448 */  
if (ps2_wide_squeeze) xscale *= PS2_WIDE_SQUEEZE_X;
```

This baseline 4:3 correction and the 16:9 anamorphic squeeze use the same mechanism. The 3D pass post-multiplies projection; the 2D pass applies a matching centered transform. World-anchored 2D overlays stay aligned only if both passes use the same scale.

Aspect correction and placement are separate jobs. The pass scale corrects geometry; HUD layout must re-anchor edge positions with the inverse scale (`hud_ar_x()` in the game). Centered elements need no horizontal correction.

Any pass that replaces matrices inherits both jobs. Attract overlay passes load their own projection/view, so they must re-apply the squeeze and re-anchor edge-based screen coordinates before adding half-size offsets.

"Full-screen" fills are not simply `(0,0,SCREEN_W,SCREEN_H)` inside the PS2 2D aspect pass. Horizontally, use the inverse AR span from `hud_ar_x(0)` to `hud_ar_x(SCREEN_W)`. Vertically, overscan past `SCREEN_H` because the GS fill rule can drop the exact boundary row. Let the GS scissor clip the extra pixels. The same horizontal span rule applies to tiled texture backdrops.

Chapter 7 - ps2gl State, Batching, and Frame Boundaries

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for the drawing environment and per-primitive state, and Ch 4 (Image Data Transmission) for the GIF packet formats a batch ultimately becomes; VU User's Manual for the VU1-side nature of ps2gl's transform/submit paths.

The single most useful performance instinct on this hardware: stop counting vertices and start counting state changes. ps2gl turns a render-state transition or texture bind into a re-sent GS environment packet, which dwarfs the cost of the handful of vertices it surrounds. This chapter is the batching discipline that follows from that, plus the frame-boundary rules that keep the swap path from faulting.

On ps2gl, state transitions and texture binds often cost more than vertices. Batch by state first. A tiny quad that reconfigures the full pipeline can be more expensive than dozens of vertices under one already-correct state block.

The text path proved it in stages:

- per-glyph `DrawTexturePro` was about 70 full batch setups for the gameplay HUD;
- batching each string saved several milliseconds;
- batching across strings saved more by avoiding per-string font texture binds;
- batching HUD rects was the real win, because each previous rect reset the full GL/GS pipeline for four vertices.

General rule: hot paths should use `begin/add/end`, hoist state into `begin`, and vary only uniform color or data inside.

Keep one uniform color per `r1Begin / r1End` batch. ps2gl has no reliable unlit varying-color renderer. Mixed colors inside one batch can fault, hang, or route into an unsupported renderer. `r1End` alone does not necessarily flush; adjacent same-mode/same-texture batches can merge. If colors vary per primitive, either flush each primitive or bucket by quantized RGBA and emit one uniform-color batch per bucket.

This applies to 2D and 3D. Speedlines bucket by alpha. Particles bucket by quantized RGBA. Lens flare ghosts have few distinct colors, so one flush per ghost is already fine.

Do not call raylib or timer APIs immediately after `EndDrawing()` on PS2. The raylib4ps2 `EndDrawing()` path flushes and swaps, and the main loop expects GS housekeeping immediately afterward. A `GetTime()` call in that narrow slot faulted. Measure logic and render before `EndDrawing()`, then derive swap from frame time if needed.

Avoid `%f` formatting on PS2's libc in per-frame HUD/debug strings; it has crashed. Use integer fixed-point formatting such as `%d.%02d`.

Chained translucent passes should share one state block. If water splash and engine jets both want lighting off, depth writes off, alpha blend on, `PGL_CLIPPING` off, and edge AA off, the caller should set that state once, call both helpers, and restore once.

Public PS2 draw primitives should fence their own state. Internal add calls between begin/end can assume the begin state, but public entry points must not depend on what the previous pass happened to leave behind.

Chapter 8 - Depth, Blending, and 2D Primitives

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) — the ZTST pixel test, alpha-blending equation and its $0x80 = 1.0$ factor convention, the texture environment (MODULATE), and the rasterization fill rule all live here.

Two GS facts drive this whole chapter and are worth holding in mind: the GS has no separate "depth test enabled" bit (disabled depth simply is `ZTST=ALWAYS`), and its alpha-blend factor treats 128, not 255, as 1.0. Almost every 2D and overlay bug below is one of those two facts leaking through an OpenGL-shaped API that pretends otherwise.

`glDisable(GL_DEPTH_TEST)` is unreliable on ps2gl for overlay passes. ps2gl's cached flag can say depth is already disabled while the GS draw environment still has a real Z test. For depth-independent overlays, force the pass function instead: enable depth test, set `glDepthFunc(GL_ALWAYS)`, disable depth writes, draw, then restore `GL_LEQUAL`. On the GS, "disabled" depth is effectively an always-pass mode anyway.

Untextured alpha-blended primitives do not fade correctly. ps2gl scales untextured vertex alpha by 255, while the GS blend factor treats 128 as 1.0. Upper-half alpha values clamp or over-blend. If something must fade, make it textured with a 1x1 white texture under `GL_MODULATE`.

raylib's 2D shape functions are unsafe or slow on PS2. They often emit untextured per-vertex color, which hits the missing unlit renderer. Use a PS2 shape helper that binds a GS-alpha-scaled 1x1 white texture, uses a single uniform color, disables culling, forces correct alpha blend, and flushes so it does not merge into a neighboring varying-color batch.

Do not use raylib's built-in shapes texture for alpha-blended PS2 fills unless it has been GS-alpha-corrected. A 255-alpha 1x1 white texture saturates the GS alpha scale and turns dim quads fully opaque. Also force 1x1 textures to nearest filtering; hardware linear filtering can sample adjacent VRAM outside a 1x1 texture even when PCSX2 appears fine.

The GS quad diagonal matters. Textured quads split along one diagonal showed a 1-pixel seam on glyphs. The working vertex order for raylib's `DrawTexturePro` is `BL, TL, TR, BR`, which makes ps2gl's quad shader produce the cleaner diagonal. Integer snapping, half-pixel offsets, explicit triangles, and the old OpenGL 0.375 translation did not fix it.

The old desktop OpenGL `glTranslatef(0.375,0.375,0)` hack is unnecessary on PS2. The GS rasterization convention differs; keep the hack removed.

Chapter 9 - Geometry, Clipping, Projection, and Display Lists

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) and Ch 4 (Image Data Transmission) for what reaches the rasterizer; VU User's Manual and VU Instruction Manual for the VU1 transform/clip path ps2gl uses; EE Overview Manual Ch 3 (Functional Overview) for the EE→GS data flow the software clipper participates in.

The PS2 has no hardware near-plane clipper the way a desktop GPU does — geometry that crosses the view plane or runs off the fixed-point coordinate range does not get safely discarded, it wraps into garbage. That single absence is why this chapter exists: every working 3D path here is really an answer to "who clips this, and against what guard band," whether that is the EE software clipper, a proven bounding-sphere bypass, or a whole-triangle cull.

Raw world-space `glBegin` inside `BeginMode3D` does not work reliably on raylib4ps2. raylib's camera matrix and ps2gl's current `GL_MODELVIEW` can be desynchronized. Use one of the proven routes:

- display-list models via rgl transforms and `glCallList`;
- software-clipped meshes through `DrawMeshSH` or `DrawMeshSHNear`;
- screen-space/2D effects after `EndMode3D`;
- special eye-space raw GL for perspective textured quads with identity modelview.

For perspective-correct rotating textured quads, use the `DrawMeshSH` recipe: `BeginMode3D` to set projection, flush rgl, set modelview identity, submit eye-space vertices with raw `glBegin(GL_TRIANGLES)`, and let projection produce the W used for STQ interpolation. 2D screen-space quads interpolate texture coordinates affinely and wrinkle when tilted.

`SHBasis` must track camera roll, not just yaw and pitch. Build it from camera vectors:

```
forward = normalize(target - position);
right = normalize(cross(camera.up, forward));
up = cross(forward, right);
```

This keeps software-clipped geometry aligned with raylib's camera and `GetWorldToScreen`, especially during banked flight.

`PGL_CLIPPING` has two regimes:

- raw display-list models need it on by default, with a bounding-sphere bypass only when the whole model is provably in front of the near plane and inside a tight guard band;
- software-clipper meshes have already been clipped on the EE, so `PGL_CLIPPING` should be off or ps2gl can still reject distant valid draws.

Particles are not an exception. With `PGL_CLIPPING` on, distant explosions thin out. But a particle is one triangle, so do not run the full software clipper on it. Do a cheap whole-triangle near plus four-side guard-band cull, submit raw, and drop any particle crossing a plane. Defer `r1Begin` until the first survivor so an all-culled bucket never emits a zero-vertex GIF packet.

Projective effects must be bounded. Anything using `1/zc` can explode as `zc` approaches zero. PC and DC viewport clipping can hide that; PS2 GS coordinates wrap into garbage. Clamp projected offsets, fade near singularities, and reject NaN/Inf by bit inspection because `-ffast-math` can delete ordinary non-finite guards.

Display lists capture texture handles at compile time. They do not keep the source `Model` alive, but `UnloadModel` frees the textures the list references. Either keep model textures live for the list lifetime or delete/recompile the display list when loading/unloading the model. In the case study, the logo follows the PC/DC scene lifecycle by deleting the compiled list before unloading the model and recompiling when the model returns.

Display-list model culling is asset-specific. The F22 uses reversed winding and needs explicit culling state. Enemies mirror the player install-once shape but use a simpler uniform tint path instead of per-stage lighting.

Stream per-frame generated geometry through `begin/add/end` helpers. Do not pack several kilobytes of temporary primitive data on the EE stack before calling the backend; the stack can corrupt and surface as an instruction-fetch TLB exception. The water splash tooth emitter is the canonical pattern: begin sets state and resets a cached mesh, add appends each tooth, end clips and draws once.

Flat per-vertex color needs a faked flat light. ps2gl has no unlit per-vertex-color renderer — every `PerVtxMaterial:kDiffuse` microprogram also requires `Lighting:1`, and per-vertex color is only transferred to VU1 when `GL_COLOR_MATERIAL` is enabled. With color-material off you silently get the last `glColor` for the whole batch; with it on but lighting off the lookup finds no renderer and faults. To render flat per-face colors (a procedural city of per-color building faces) without real shading, give every vertex the same constant normal and a single directional light pointing along that same eye-space normal, so $N \cdot L \equiv 1$ everywhere and the diffuse term passes the vertex color straight through unshaded. Use `glColorMaterial(GL_FRONT_AND_BACK, GL_DIFFUSE)` (only `GL_DIFFUSE` varies per vertex on ps2gl), zero ambient and specular, and configure the light under an identity modelview so its direction matches the emitted normal. The color must also survive the EE clipper, so the clip vertex carries RGBA and interpolates it across cuts.

Chapter 10 - Texture Formats, Alpha, and Filtering

Relevant manuals: GS User's Manual Ch 2 (Local Memory) for the PSMCT32/16/8/4 pixel-storage modes, CLUT storage, and page/block geometry; Ch 3 (Drawing Function) for texture mapping, the TEXA alpha expansion, and filtering; Ch 8 (Details of GS Local Memory) for addressing; `ps2textures.htm` as a quicker companion to the PSM/CLUT tables; GS User's Manual Supplement for errata.

GS texture work is a VRAM-residency problem first and an image-quality problem second. The GS has 4 MB of local memory carved into fixed-size pages, and a texture is only cheap if it fits and stays resident; the choice of pixel-storage mode (32-bit vs 16-bit vs 8-bit paletted) is mostly a choice about how many pages you spend and therefore what stays resident. This chapter is that economy, plus the alpha and alignment traps that bite once you leave plain RGBA32.

Size textures to on-screen footprint and GS residency, not source resolution. The expensive case is not "large texture" by itself; it is a texture too large to stay resident, causing eviction and re-upload on the draw that needs it. Oversized HUD atlases both cost upload time and evict other textures.

Use console-scale art as the default PS2 source, not HD sources. HD PNGs can decode to many megabytes in EE RAM and fault when an image decoder walks past allocation failure. Keep the original aspect unless intentionally changing art.

PSMCT16 halves VRAM for opaque art. A 512x512 RGBA32 texture is 128 GS pages and does not fit the default 64-page largest slot; 16-bit 5551 is 64 pages and fits. Pack in GS bit order: red in the low five bits, then green, blue, alpha in bit 15. Remember the cost: only one alpha bit. Cutout sprites, fonts, and smooth alpha art should not use this path.

PSMT8 paletted textures are the practical workhorse. They use 8-bit indices plus a 256-entry 32-bit CLUT, giving quarter-size texture bodies with full CLUT alpha. The case study eventually moved the full PS2 texture set to PSMT8 at full console-target resolution.

PSMT8 rules:

- reorder CLUT entries for GS CSM1 layout at build time;
- pack opaque CLUT alpha as `0x80`, not `0xFF`;
- make each texture own its CLUT in the ps2gl fork, because a single global CLUT causes two paletted textures to sample the last uploaded palette;
- reserve kPsm32 GS slots for CLUTs, because `CMMClut` allocates a 16x16 kPsm32 area and cannot see a slot declared as PSMT8;
- all mip levels of one PSMT8 texture share the base texture's one CLUT.

Every DMA-fed embedded blob needs 16-byte alignment. When multiple texture blobs were packed into one generated `.s`, blobs after a 4-byte size field landed at address 4 mod 16. The EE DMAC truncated the source address to a qword boundary and the CLUT upload read four bytes early, shifting the palette by one entry and turning index zero black. Emit `.balign 16` before every blob, not just once per file.

Smooth color ramps are poor palette-alignment tests. Use a sharp grid probe with distinct colors at neighboring entries. If PCSX2 hardware and software agree on wrong colors, suspect data/alignment,

not sampling.

Texture alpha stored in GS memory needs the GS scale. Procedural alpha textures should run through `ps2_image_alpha_to_gs_range()` before upload. Runtime `glColor4ub` is already normalized by ps2gl; do not rescale runtime colors again unless bypassing ps2gl.

Texture filter and wrap state changed over the life of the fork. Current rule: state every asset's filter explicitly. Point-sampled font atlases and 1x1 solid textures should be nearest. Bilinear 1x1 textures can flicker on hardware. When using raw `glTexParameteri`, pass ps2gl's `GL_REPEAT` and `GL_CLAMP` symbols, not desktop numeric constants; ps2gl's enum values differ.

Skybox and floor UV magnitudes matter on real GS hardware. Large absolute UVs cause wobble because the GS perspective-correct interpolation loses precision as S/T magnitude grows. Keep UVs local to slices or wrap the integer tile count out of scrolling UVs. PCSX2 can hide this.

Anamorphic texture storage is a valid VRAM lever: store one axis squeezed and let GS sampling stretch it back. The loader must report logical dimensions, not stored dimensions, or any code deriving UVs, sprite rects, or atlas frame sizes will compensate in the wrong direction.

Chapter 11 - Manual Mipmaps, Dither, and GS Texture Precision

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for texture LOD selection, mip filtering (MMIN/MMAG), and dithering (DIMX/DTHE); Ch 7 (Registers) for the TEX0/TEX1 fields (MXL, MMIN, K, L, LCM) and the MIPTBP1/2 mip base pointers.

Mipmaps on this stack are a build-it-yourself feature: the GS hardware does them natively through TEX1 and MIPTBP, but ps2gl ships no path to drive those registers, so this chapter is the manual upload recipe. It pairs with dithering because both are about fighting the precision loss of a 16-bit framebuffer — one in texture LOD, one in the final store — and both fail in instructive, hardware-only ways that an emulator renders perfectly.

ps2gl did not ship a mipmap path. Real GS mipmaps require:

- `TEX1.MXL` and `TEX1.MMIN` on the texture's environment;
- explicit `MIPTBP1` and `MIPTBP2` base pointers for levels;
- per-level resident textures;
- locked mip slots so ps2gl's allocator does not reuse them;
- an LOD bias in `TEX1.K`, often strongly negative for grazing floors because the GS has no anisotropic filtering.

MIPTBP addresses need not be contiguous. Upload each mip as its own resident texture and harvest each GS base pointer. Do not rely on automatic contiguous mip addressing.

MIPTBP is global enough to bite. With more than one live mipmapped floor, the new texture can stomp the previous texture's mip pointers. The case-study policy: only one mip texture live at a time, release the old pyramid before creating the new one, and deep-unload floors during stage changes.

The GS supports trilinear (`MMIN = 5`) in one pass. The Dreamcast/PVR trilinear black-surface rule does not apply to GS. But real hardware blends mips with limited precision. PCSX2 can show perfectly smooth trilinear gradients while the real GS shows faint stepped bands. Use a mip-tint probe to verify whether MIPTBP/MXL/MMIN are working and where transitions land.

A 16-bit framebuffer needs GS dithering. The GS has DIMX plus DTHE for this, and ps2stuff's draw environment already has a default matrix. Enable dither through the drawenv object, not a raw register write, because ps2gl re-sends the drawenv and will clobber ad hoc writes. Gate the effective value to 16-bit layouts.

Texture-space dither does not fix magnified framebuffer banding. The sun disc case showed why: bilinear magnification low-passes texel-frequency dither back into a smooth ramp, then the 16-bit framebuffer quantizes it again. Screen-space dither or fewer quantizing additive passes may help, but the baked texture dither experiment was reverted.

Chapter 12 - GS and EE Memory Management

Relevant manuals: EE Core User's Manual for the R5900 caches and memory system (why IOP-DMA'd data needs the uncached mirror); EE User's Manual for the DMAC and scratchpad; GS User's Manual Ch 2 and Ch 8 (Local Memory) for the GS page/slot layout the VRAM allocator manages.

There are two separate memory budgets on a PS2 and they fail differently: EE RAM (32 MB, where decoded assets and the geometry buffers live) overflows into TLB faults, while GS VRAM (4 MB of paged local memory) overflows by silently failing to make a texture resident. This chapter is about measuring both honestly — and the non-obvious facts that textures are double-resident by default, and that for VRAM the number that matters is the largest *single* free slot, not the total free page count.

Audit raylib4ps2's immediate buffer first. `pglInit(128*1024, ...)` allocates about 16 MB across double-buffered immediate geometry arrays. The case study reduced this to `32*1024` qwords and reclaimed about 12 MB of EE RAM while keeping large headroom. Display-list models do not use that immediate buffer; software-clipped raw geometry does.

Use `mallinfo()` for live EE heap readings. On PS2, `uordblks` and `arena` tend to be close because the heap does not return pages to the system, but it is still useful live data.

Textures are double-resident by default: their pixels live in EE RAM as DMA sources and in GS VRAM after upload. This is useful for lazy re-upload after eviction, but it costs memory. If reclaiming EE memory becomes important, be explicit about how an evicted texture can be rebuilt from embedded data.

Add a real GS VRAM query in the fork. `pglPrintGsMemAllocation()` is useful but stdout-only. The valuable live numbers are total pages, used pages, and largest free slot. The largest free slot matters because a texture must fit in one slot. Page count alone does not answer residency.

Default GS slot maps are as important as texture byte size. A 512x512 texture is 32 pages at 8-bit, 64 pages at 16-bit, and 128 pages at 24/32-bit. If the largest free slot is 64 pages, full-res 24/32-bit cannot fit regardless of total free pages.

Chapter 13 - Effects and Scene-Specific Rendering

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for perspective- correct texturing, alpha blending, and the precision behavior behind UV wobble; VU User's Manual for the transform path. Case-study specifics come from the development notes.

This chapter is the most game-specific in the handbook, but the techniques generalize: how to couple sky/floor/haze geometry so pieces never poke through each other, how to keep projective effects (speedlines, lens flare) from exploding near their $1/z$ singularities, and how to keep UV magnitudes small enough that the GS's reduced-precision perspective division does not visibly swim. Treat the specific effects as worked examples of those three ideas.

Skybox, floor, haze, and water-wall geometry are coupled. Keep them on the same polygon chord endpoints and slice assumptions so haze or floor pieces do not poke past the sky cylinder. The PS2 helper meshes bake absolute floor Y; the world anchor moves them in X/Z only. Do not subtract `world.origin.y` a second time.

The underwater wall's vertical scroll is intentional. It reads as rising current. Keep the under-cap and wall as separate meshes/draws because the cap uses floor forward scroll while the wall adds its own vertical scroll.

Translucent haze ports cleanly through the software clipper. Build CPU-side mesh arrays, use GS-scaled texture alpha, draw after opaque environment, and avoid exact-coordinate z-fighting. On PS2 the haze renderer is its own clipped path; on Dreamcast a different depth-state rule may apply.

Speedlines need PS2-specific projection discipline. Derive the vanishing point from player yaw, manually project with signed `zc`, clamp the drawn VP offset, fade based on horizontal offset near the yaw singularity, and reject non-finite coordinates by bit inspection. The discontinuity at 90 degrees is mathematically real; hide it with fade rather than using `abs(zc)`, which puts the VP on the wrong side.

Speedline alpha is texture-modulated on PS2. Because varying vertex colors are unsafe, bake the tail-to-head ramp into a texture and use uniform alpha per quad/bucket. Match head brightness to the cross-platform path and accept the slightly brighter tail.

Lens flare occlusion should test the sun disc, not only the center. Sample the center plus cardinal perimeter points, ray-test against model OBBs, and scale flare alpha by visible sample fraction. Spheres over-occlude long thin models; world AABBs fail when the model rotates. Inverse-rotated OBB slab tests match the model orientation and avoid rebuilding boxes.

Particles need both sim and renderer policy. Simulation uses priority eviction so explosions beat sparks and sparks beat trails. Rendering uses PGL clipping off, whole-triangle culling, white texturing for alpha fade, quantized RGBA buckets, and a no-empty-batch guard.

A fixed-step inner loop must restore every global it borrows. The attract recorder restored `game.time` but not `game.dt`, causing the cinematic camera to run at frame speed after deterministic sim ticks. On 60 Hz consoles the bug was invisible; on uncapped PC it shifted choreography and made the PS2 renderer look guilty for missing bullets that no longer existed at that timestamp.

Clamp bad `dt` values immediately after reading frame time. Use `if (!(dt > 0.0f) || dt > max)` so NaN and negative values reset too. NaN state is sticky and can poison cameras, positions, and debug displays permanently.

A sweeping searchlight cut by the world is a bake problem, not a raytrace problem. The naive shape — a light cone terminated at the first wall it strikes — is a raycast per beam per frame, and on a streaming world that cost detonates on the admission edge: the frame that pulls a new window into view pays for every new candidate at once. Memoizing "once per building" does not help, because a window admits hundreds of buildings in one frame; moving the scan from the stream build to a gather at ground rebuild only relocates the same burst. The fix is to precompute everything the camera cannot change and leave the runtime only the part it must. Bake the light placement, the sweep amplitude, and — because the billboard facing is unknowable offline — a full 360-degree gather of the buildings the cone could ever intercept, cleared down to a handful by a conservative height heuristic and stored as small record indices into the existing world bake. At runtime each beam does only a 2D slab test of its two cone edges against those few candidates and reshapes the primitive. Never bake a lookup table of a camera-dependent quantity (a pre-cut length that assumed a facing is a stale-bake bug by construction); bake only invariants, and compute the facing, the sweep phase, and the cut live.

Billboard by the camera's orientation, not its position. Position (radial) billboarding gives every object its own yaw, so neighbors disagree and each sprite visibly rotates on its axis as the camera strafes; orientation (view-plane) billboarding shares one facing across the whole field and costs a single normalize per frame rather than one per object. It is also the sprite-correct look. A corollary for planar effects: a fan spread across azimuth lies in a slanted plane and goes edge-on invisible from common angles, whereas a vertical billboarded plane reads from every side — and its intersection with a vertical prism is exactly a 2D rectangle, which turns the visibility cut into a flat slab test rather than 3D geometry.

An offline bake shared with the runtime needs a single-sourced contract and a provably conservative prune. Put every constant both the baker and the game depend on in one header included verbatim by both, so a tuned value cannot drift between producer and consumer (the same stale-profile hazard that a hardcoded second copy always eventually causes). Make the baker's coordinate decode match the game's bit for bit, including any world-space mirror or height conditioning. And when the bake prunes candidates with a heuristic gated by a runtime knob, prove the bound against the knob's ceiling, not its default value — a cone that can widen at runtime must still find every blocker the bake kept, or beams punch through buildings that were cleared offline.

A long thin additive primitive is the inverse of a particle quad. Small screen-facing quads are culled whole and never clipped, but a beam is large and must be clipped — just not by the wrong clipper. It cannot ride a raw no-clip path (coordinates past the GS 2047 window wrap into garbage) and it cannot ride a fixed-output-cap clip microcode (an oversized clipped fan is silently dropped, the same reason large land polygons stay on the EE clipper); route it through the EE software clipper, whose cost at these vertex counts is negligible. Texture it, too: untextured vertex alpha does not fade on the GS (untextured color caps at 255 versus 128 textured), so an additive gradient renders as a flat slab unless it is textured. And guard the empty batch, because a GIFTag with NLOOP zero hangs the GS.

When the same effect is re-expressed in a second, immediate-mode renderer, two portability traps surface that the PS2 path had silently handled. The first: an immediate-mode draw pass inherits whatever global render state the previous pass left set — cull, blend, depth-test, depth-write all leak

forward. A pass that draws view-dependent-winding geometry (billboards, camera-facing cards) must therefore set its own cull state, because from the wrong side every triangle is back-facing and silently culled. The PS2 beam pass had never shown this because it explicitly disables face culling for its own draw; that explicit prologue has no automatic counterpart when the pass is rebuilt in a different API, so port the state calls too — they are part of the pass, not boilerplate. The fastest way to localize such a bug is a fullbright sentinel: draw one unmistakable triangle dead-ahead through the same path. If the sentinel shows but the real geometry does not, the emit is empty (a data problem); if even the sentinel vanishes, the draw state is killing pixels (a state problem) — one run splits the two.

The second trap concerns data shared across renderers. A cache that snapshots a per-frame offset and cancels it with a matching compensation applied elsewhere is correct only while every consumer honors that unstated pairing. HyperSolar's ground stack bakes each rebuild's floor drift into its vertices and cancels it by shifting the draw camera's eye; the PS2 software clipper takes an explicit per-pass eye and can participate, but a fixed-view renderer that locks one view matrix per pass cannot, and fixing only half the pipeline re-creates the error inverted (the earlier PS2 double-shift, where emit moved to live drift while the draw still applied the compensated eye). The durable cure is to make the data self-describing rather than paired: store in a stable frame, apply the live per-frame delta at the single point of emit, and hand every renderer a plain camera. It works here because the beam's cut math is relative — cone edges against building corners — and therefore translation-invariant, so it runs in the stable frame unchanged. Prefer data that carries its own frame over data that needs a correction applied somewhere else.

Chapter 14 - Input, Debug UI, and Save Data

Relevant manuals: EE Overview Manual Ch 2 for system context (the IOP owns pad and memory-card services, the EE reaches them over SIF RPC). libpad and libmc behavior is ps2sdk-specific; the rules here are practical, from the case study.

Input, the debug UI, and save data are grouped because they share one theme: they all cross the EE↔IOP boundary through libraries that are unforgiving about buffer placement and init ordering. The recurring failure here is not a wrong value, it is a DMA writing into a stack buffer and corrupting a return address — so the rules are mostly about *where* memory lives (file-scope, aligned, uncached) and *when* you initialize (before raylib touches SIF RPC).

Map libpad into the shared `PadState` early. Cross, Circle, Square, Triangle map to A, B, X, Y in the xbox-style game abstraction; d-pad, sticks, and digital triggers fill the shared fields. Keep raw libpad state for PS2-only debug inputs such as SELECT or shoulders and for the input debug overlay.

Debug menu platform rows should be platform-real. FPS target is `PC_ONLY`, not PS2. PS2 is display/vblank locked. PS2's Graphics submenu owns video mode 448i/448p, aspect 4:3/16:9, anti-aliasing, dithering, the interlace flicker filter (interlaced modes only), screen fit/pos where implemented, and memory-card save target.

GS edge AA on opaque models can read as a black cel outline rather than soft anti-aliasing. Default it off and expose it as a debug option rather than assuming it is a visual improvement.

For memory-card saves, initialize libmc before GL/raylib init. `mcInit()` calls `sceSifInitRpc(0)` unconditionally; running it late can desync ps2link fileio. Load `rom0:SIO2MAN`, `rom0:MCMAN`, and `rom0:MCSERV`, then `mcInit(MC_TYPE_MC)`. Use the plain modules, not the `X*` variants.

libmc result and transfer buffers must be file-scope and 64-byte aligned. The RPC DMA writes into them; stack buffers can corrupt return addresses and surface as instruction-fetch exceptions.

Decode `mcGetInfo` and `mcSync` results carefully. `0` and `-1` can both mean an OK card state for this flow. `-2` is unformatted, and values below about `-10` indicate no card. `mcClose` commits through MCMAN's cache; successful write plus close persisted across power-off in hardware tests.

Under ps2link, libmc RPC bursts can leave the next unrelated fileio call broken. A single blocking stdout write heals the shared ps2link path. This is a dev-loader artifact, not a real hardware save bug, but centralize the barrier so it is understood rather than scattered as mysterious prints.

A BIOS-browsable save needs a directory with `icon.sys` and an `.icn` model. Missing either appears as "Corrupted Data." The icon model is a tiny animated 3D format with 16-bit fixed-point positions/normals/UVs and a 128x128 BGR555 texture. Write it once and avoid rewriting it on every save.

For `.icn` geometry, follow the BIOS camera conventions:

- the BIOS/mymcplus reference effectively negates Y and Z on load;
- the icon rotates/zooms around model point `(0, 2.5, 0)`;
- written Y around `-2.5` centers the model on that pivot;

- keep geometry tiny, such as a textured cube or card;
- wind faces CCW outward;
- flip the texture 180 degrees.

The case study's GLB-to-ICN converter captures those rules.

Chapter 15 - SPU2 SFX Fundamentals

Relevant manuals: SPU2 Overview Manual Ch 1 (Overview of SPU2) for the ADPCM waveform format and 2-core/24-voice layout; Ch 2 (Sound Generation) for voice generation, the PITCH math (§2.1.2), ADSR envelopes, and mixing/VMIX; Ch 3 (Register List) for the VOLL/VOLR/PITCH/ADSR/KON register fields.

The SPU2 is a local-memory ADPCM synthesizer, not a stream player — it does one thing extremely well (per-voice pitch and volume on samples resident in its 2 MB of RAM) and nothing else. This chapter is how to feed it: the IOP-address gotcha in the upload path, the encoder quality gap that makes a naive ADPCM tool sound like AM radio, and the voice-pool and block-flag conventions that separate a looping layer from a one-shot. Streaming long music is hard enough to get its own chapter (Chapter 16).

Use `libsd/ps2snd` direct voice control for game audio that needs continuous runtime pitch and per-voice register control. `audsrv` is not the right abstraction for that job.

Load both `libsd.irx` and `ps2snd.irx`, in that order. `rom0:LIBSD` is not the EE-facing RPC server, and `libsd.irx` alone is not enough. `sceSdInit` binds to the PS2SND RPC registered by `ps2snd.irx`; without it, the bind loop hangs.

`sceSdVoiceTrans` takes an IOP address as source. EE pointers are meaningless to the IOP module. Allocate an IOP buffer, DMA EE data to IOP, then ask `libsd` to DMA IOP data to SPU2. The SPU destination argument is the SPU byte address cast as a pointer value, not the address of a local variable containing that byte address.

Do not chunk through one reused IOP bounce buffer unless the previous IOP to SPU DMA has fully completed. Reusing it early overwrites in-flight audio data and sounds like gritty radio noise. The simple correct path for resident SFX is one IOP buffer sized for the whole sample, one EE-to-IOP DMA, one `sceSdVoiceTrans`, wait with `sceSdVoiceTransStatus`, then free the IOP buffer.

SPU2 pitch is native and should be used. For 48 kHz samples, `PITCH = 0x1000` means unity playback. The usable range is `1..0x3FFF`, roughly -12 to +2 octaves. Encode samples at 48 kHz so unity pitch is exact.

For continuous loops whose amplitude is driven by game mix code, use instant attack and flat full sustain ADSR so `VOLL` / `VOLR` are the actual level. Update pitch and volume per frame as needed.

SPU2 voices are PS-ADPCM. The stock `ps2adpcm` encoder sounded poor because it lacked noise shaping and robust predictor handling. A good encoder should choose best filter/shift per block, noise-shape quantization error, keep predictor history continuous across blocks, and clamp input peaks to avoid overflow.

Loop and one-shot ADPCM block flags differ:

- loops set first block `0x06` and last block `0x03`;
- one-shots set only the final block `0x01` END flag.

One-shots should use a round-robin voice pool. Re-keying a still-playing voice is an acceptable evict-oldest policy. In the case study, loops use voices 0-3, one-shots use 4-11, and music uses 22/23.

Keep shared mix math shared. The case study moved target volumes, smoothing, ducking, pan, and SFX suppression into common code so PC/web, Dreamcast, and PS2 differ only at the leaf "set this voice/stream" layer.

Chapter 16 - Music Streaming and Custom IOP Modules

Relevant manuals: SPU2 Overview Manual Ch 2 (Sound Generation) for the stream input model, the loop/repeat block flags, and VMIX mixing; EE Overview Manual Ch 2/3 for the EE↔IOP split and SIF RPC; EE User's Manual for the DMAC paths the transfers ride on.

Music is where the SPU2's "not a stream player" nature collides with reality: a song is too big to fit in SPU2 RAM, so you must continuously refill the half of a ring buffer the voice is not currently reading. The hard-won conclusion of this chapter is architectural — that refill loop belongs on the IOP as a custom IRX, not on the EE render thread — and getting there means writing your first IOP module and learning why the stock streaming API deadlocks.

Do not use ps2snd's stream API for music in this project shape. `sndStreamOpen` deadlocked the IOP on both `host:` and legacy `mass:`. Its model opens files on the IOP inside blocking RPC/stream machinery, which collides with ps2link fileio and lazy USB mounting.

The working design is a custom SPU2 ring streamer:

- two voices, one per stereo channel;
- each voice uses a two-half ring in SPU2 RAM;
- the file is chunk-interleaved `[L chunk][R chunk]...`;
- block flags make the ring self-loop;
- `NAX` tells which half the hardware is currently reading;
- refill the half the play cursor just left;
- rewind at EOF for looping tracks;
- keep VMIX bits routed into the dry mix.

Run the streamer on the IOP, not the EE. An EE-side streamer worked but caused frame hitches and GS margin artifacts during screen changes because file reads and SPU DMAs touched the render thread. Moving file I/O and SPU DMA to a custom IRX made the EE send only play/stop/volume/pause RPCs and eliminated per-frame audio work.

The IOP module's RPC handler should not do blocking file reads or SPU DMA. It should copy the request, signal a stream thread, and return after consuming the command. The stream thread opens, primes, polls `NAX`, reads, and transfers. Poll with a short `DelayThread`; avoid SPU IRQ file I/O.

Music and SFX share core-wide VMIX registers. Both owners must OR their bits in without clearing the other side. In the case study, the EE SFX VMIX accumulator is seeded with music voices 22/23, while the IOP read-modify-writes to preserve SFX bits.

Custom IRX build traps:

- build with the IOP toolchain and run through `iopfixup`;
- provide your own `irx_imports.h`;
- import `memcpy` / `memset` from `sysclib` when using `-nostdlib -fno-builtin`;
- read files on the IOP through `iomanX_*`, not EE `fileXio`;
- avoid `*/` inside block-comment prose in import headers.

For USB, use the BDM/iomanX stack and the `mass0: device: iomanX.irx`, `usbd.irx`, `usbmass_bd.irx`, `bdm.irx`, and `bdmfs_fatfs.irx`, in order. This is not legacy `usbhdfs` `mass:`. The BDM stack mounts eagerly; a failed open usually means "not mounted yet", so retry.

If the EE reads from iomanX instead, add `fileXio.irx`, call `fileXioInit()`, and use `fileXio*`. Define `NEWLIB_PORT_AWARE` before including `fileXio.h` if needed and do not mix those descriptors with `newlib` `open()`.

When reading memory the IOP DMA'd into EE RAM, use the uncached mirror `addr | 0x20000000`. The EE data cache may still hold old lines.

Plain burned data CD/DVD media are illegal media on stock or FMCB-only consoles.

`sceCdGetDiskType()` reports illegal media and reads return zeros. `cdrom:` via `cdfs` is useful for authenticated discs, modchip scenarios, or emulator ISOs, not for unmodded network-boot dev. Use USB or HDD for dev removable storage.

Chapter 17 - Validation Doctrine

Relevant manuals: all of them, but this chapter is mostly empirical — it is about how to *use* the manuals and the emulator together rather than any single register.

If the handbook has one meta-lesson, it is this chapter: a PS2 emulator and a PS2 console are different witnesses, and the gap between them is exactly where the expensive bugs live. The discipline below — validate the right things on hardware, prove an object exists before debugging why it "disappeared," compare PCSX2's hardware and software renderers to localize data-vs-sampling bugs — is what turns a multi-day ghost hunt into a one-screenshot diagnosis.

Validate PS2 video modes on hardware. PCSX2 can hide bad `magH`, color/Z page mismatches, overscan position, and display-register mistakes.

Validate texture precision and filtering on hardware. PCSX2 can hide skybox UV wobble, real GS trilinear quantization, and 1x1 linear-filter VRAM fetch artifacts.

Validate standalone boot separately from ps2link boot. ps2link can mask missing IOP reset/SBV setup and can introduce its own memory-card/fileio interop hazard.

When a renderer "loses" an object, first prove the object exists at that instant. The attract bullet investigation burned time in clipping and draw-state probes when the timeline bug meant the bullets were no longer being submitted.

When diagnosing GS margin flashes, distinguish framebuffer contents from BGCOLOR. If the flash appears only outside the active 640x448 frame, fix BGCOLOR timing rather than clears or fullscreen fills.

When diagnosing color/palette corruption, compare PCSX2 hardware and software renderers. If both match the same wrong output, suspect uploaded data, DMA alignment, or CLUT binding.

When optimizing, bisect by pass before inventing abstractions. Text looked like the HUD hog; rect pipeline reconfiguration was the larger cost. The GS and ps2gl are often state-bound, not vertex-bound.

Do not publish speculative lessons as proven facts. Keep a working engineering log if you like, but promote entries into a public manual only after they have survived real validation.

Chapter 18 - ps2gl and ps2stuff Fork Delta

Relevant manuals: GS User's Manual Ch 7 (Registers) for the register fields the fork drives — `PRIM.AA1`, `DTHE / DIMX`, `TEX1`, `MIPTBP1/2`, `DISPLAY` — and Ch 5 (CRTC) for the display state the runtime-mode helpers touch; EE User's Manual for the DMA paths; VU User's Manual for the VU1 context behind ps2gl's clipping and immediate rendering.

The preceding chapters describe scars on the game side; this one describes what had to change *inside* the libraries to make those fixes possible, and — for anyone maintaining their own ps2gl/ps2stuff fork — which of those changes look upstreamable versus project-local. Read it as both a changelog and a design critique: several of the deltas are correct-but-narrow, and the text flags where a cleaner general API should eventually live.

This chapter summarizes the fork-delta report for the case-study ps2gl and ps2stuff forks. The rest of the manual describes the scars from the game side; this chapter describes what changed in the libraries and which parts are likely useful to the broader PS2 SDK ecosystem.

The high-level delta categories are:

- build proof canaries;
- GS memory usage query APIs;
- runtime display mode and raster offset control;
- centered viewport scaling for overscan/screen fit;
- GS edge anti-aliasing exposure;
- GS dither control through the draw environment;
- per-texture CLUT ownership for PSMT8 textures;
- PSMT8 convenience uploaders;
- manual GS mipmap upload for PSMCT16 and PSMT8 textures;
- manual mip texture lifetime and release tracking;
- a Python replacement for the old VU `gasp` preprocessing step.

ps2stuff owns the GS memory allocator, so memory reporting starts there. `CMemManager::GetMemInfo()` walks locked slots and every slot list, accumulates total carved pages, counts bound or locked slots as used, and tracks the largest single free slot. The largest free slot is the number to care about. A texture allocation must fit in one compatible slot; total free pages can lie to you.

ps2gl exposes that as a C-facing wrapper:

```
extern void pglGetGsMemInfo(int *total, int *used, int *largestFreeSlot);
```

The dependency chain is:

```
application
-> ps2gl: pglGetGsMemInfo()
-> ps2stuff: CMemManager::GetMemInfo()
-> ps2stuff: CMemSlotList::AccumMemInfo()
```

For display positioning, the ps2stuff fork adds a narrow DISPLAY-only register push. The point is to shift the raster without re-sending unrelated display state such as `PMODE`, `DISPFB`, or `BGCOLOR`. The current helper writes `DISPLAY2`, matching ps2gl's active read-circuit assumption. A more general upstream API should choose `DISPLAY1` or `DISPLAY2` explicitly.

The corresponding ps2gl API is:

```
extern void pglSetDisplayOffset(int screen_x, int screen_y);
```

The dependency chain is:

```
application
-> ps2gl: pglSetDisplayOffset()
-> ps2gl: CDisplayContext::SetDisplayOffset()
-> ps2stuff: CDisplayEnv::SetDisplay2()
-> ps2stuff: CDisplayEnv::SendDisplayPos()
-> GS DISPLAY2
```

Runtime video-mode control is exposed through:

```
extern void pglSetVideoMode(int interlaced, int overscan_mode, int screen_x, int screen_y);
```

This updates ps2gl's display interlace state, reprograms FB2, and uses different display magnification for interlaced and progressive output. In the progressive path the fork uses full buffer height and `magH = 2`, fixing the hardware bug where `magH = 4` made DTV output exactly twice too wide while PCSX2 hid the problem. This API does not rebuild GS memory layouts. Applications that change dimensions or pixel formats still need to rebuild draw/display buffers and slot maps themselves.

Centered viewport scale exists as:

```
extern void pglSetViewportScale(float sx, float sy);
```

It persists `vpScaleX/Y`, folds them into the draw context's GS scale, and updates the drawenv scissor to a centered sub-rectangle. This is not a general OpenGL viewport replacement. It is a PS2-friendly screen-fit/squish primitive for overscan and CRT calibration, bounded to $0 < s \leq 1$ in the case-study fork.

GS edge anti-aliasing is exposed as a ps2gl capability:

```
#define PGL_EDGE_AA 3
pglEnable(PGL_EDGE_AA);
pglDisable(PGL_EDGE_AA);
```

The fork tracks the draw-context state and sets `GS::tPrim.aa1` when giftags are generated. This exposes the GS primitive edge AA bit. It does not solve texture shimmer, texture minification, or aliasing inside textured surfaces.

GS dithering is exposed through the drawenv object:

```
extern "C" void pgl_enable_dither(int enable);
```

The important bit is not the function name; it is the route. Dither must live in the `CDrawEnv`, followed by `DrawEnvChanged()`, so `ps2gl` re-sends the correct state. A raw register poke is temporary and will be overwritten by the next `drawenv` send. A cleaner upstream shape would likely be `pglEnableDither()` or a `PGL_DITHER` capability.

Per-texture CLUT ownership is the most direct correctness fix. The original manager-global CLUT model fails with multiple simultaneous PSMT8 textures: the latest `glColorTable()` effectively changes the palette for other indexed textures. The fork gives each `CMMTexture` an owned `CMMClut`, has `SetCurClut()` attach the palette to the currently bound texture, and has `UseCurTexture()` load that texture's own CLUT. This should be reviewed for display-list behavior and deletion edge cases, but the concept is broadly upstream-relevant.

The PSMT8 upload convenience API is:

```
extern "C" unsigned int pgl_create_index8(  
    const void *indices,  
    int w,  
    int h,  
    const void *clut  
);
```

It wraps the correct `GL_COLOR_INDEX` / `GL_UNSIGNED_BYTE` upload plus a 256-entry RGBA CLUT upload. The assumptions must be documented: the CLUT is 16-byte aligned, is already in the storage order expected by this path, and carries GS alpha conventions.

Manual mipmaps are the largest architectural change. PSMCT16 uses:

```
extern "C" unsigned int pgl_create_mip16(  
    void **levels,  
    const int *lw,  
    const int *lh,  
    int count,  
    int kbias,  
    int min_filter  
);
```

PSMT8 uses:

```
extern "C" unsigned int pgl_create_index8_mip(  
    const void **levels,  
    const int *lw,  
    const int *lh,  
    int count,  
    const void *clut,  
    int kbias,  
    int min_filter  
);
```

Both paths create a base GL texture, upload hidden resident mip textures, lock the mip slots, set base `TEX1` fields (`MXL`, `MMIN`, `K`, `LCM`, `L`, `MMAG`), and write `MIPTBP1/2`. The dependency chain is:

```
application texture loader
-> ps2gl: pgl_create_mip16() or pgl_create_index8_mip()
-> ps2gl: CMMTexture::SetMipLevels()
-> ps2gl: pgl_send_miptbp()
-> GS_TEX1 + MIPTBP1/2
```

Release uses:

```
extern "C" void pgl_delete_mips(unsigned int baseId);
```

That registry unlocks and deletes the hidden mip textures before the base texture is deleted separately. In the case-study fork the registry is deliberately small and pragmatic. A more upstreamable design should make mip levels owned by the base texture and should re-emit MIPTBP as texture state, otherwise multiple simultaneous mipped textures remain unsafe.

The VU preprocessor replacement is `vu1/gasp.py`, a Python subset of the old `gasp` macro features needed by the current VU sources. It supports includes, macros, conditional assembly forms, repeats, `.equ`, macro parameters, `\@`, and `\&var`. Regeneration is gated behind `REBUILD_VU1=1`; normal builds expect checked-in `.vsm` output. Treat this as a compatibility tool, not a full assembler preprocessor.

Canary prints are project diagnostics, not upstream candidates. Their purpose is to prove the build linked the intended fork rather than toolchain-installed archives. The same idea is useful in any project with nested forks, but the exact print strings should remain project-local.

The fork report's upstreaming read is:

- likely good candidates: memory info query, `pglGetGsMemInfo()`, DISPLAY-only position push, runtime display offset, edge AA exposure, dither exposure, and per-texture CLUT ownership;
- candidates needing design work: runtime video-mode switching, centered viewport scale, PSMT8 upload helpers, and full mipmap support;
- project-local as-is: canary prints, relative sibling include paths, fixed-size mip registry, and raw one-shot MIPTBP DMA as public behavior.

Mipmap state ownership, once the largest unresolved library-design issue, is now resolved (Chapter 21): `MIPTBP1/2` moved out of a one-shot global register write and into each texture's settings packet, so any number of mipmapped textures coexist instead of the old single-owner invariant. Two further fork deltas landed alongside it and are the strongest upstream candidates in the set — GS hardware fog wired into the vertex kernel (Chapter 19) and an explicit texture-image ownership transfer that closes the "fork keeps the texels pointer" leak (Chapter 21) — together with the interrupt-drain freeze fix, the per-vertex-alpha kernel variant, and the renderer-registry zero-init (Chapter 20).

Chapter 19 - GS Hardware Fog: The Unit Works, the Fork Never Wired It

The GS has a per-vertex fog unit and almost no OpenGL-ish fork uses it. The unit is real, it is free (it rides the vertex you were already sending), and the only reason your fork does not expose it is that whoever wrote the vertex kernel left the fog byte hardwired to zero. This chapter is how to turn it on.

The fog byte is already in the wire format. A GS vertex in the XYZF2 form carries an 8-bit F field alongside X, Y, Z. Fork VU1 kernels that pack XYZF2 already emit the qword — they just write F as zero with `PRIM.FGE = 0`, so the unit is dormant, not absent. Enabling fog is three edits, not a new pipeline: set `FGE = 1` in the giftag PRIM (which needs `PRMODECONT.AC = 1` so the PRIM fields are honored), set the global `FOGCOL` register, and write a real F per vertex in the kernel. `F = 255` is minimum fog (a $\sim 1/256$ residue remains — true no-fog is `FGE` off), `F = 0` is solid `FOGCOL`. The GS DDA-interpolates F across the triangle exactly like color.

Watch the ADC packing. F lives in the XYZF2 W-word at bits 11:4 (the ADC/kick bit is 15). Kernels that set ADC with the classic "add `0x7fff` and let the carry land in bit 15" trick fill bits 11:4 with garbage — harmless while `FGE` was off, but with fog on that garbage renders as per-triangle fog bands. Mask the ADC word to bit 15 and OR in your F (an `ftoi4` of the fog fraction is already the `<<4` packing). A red `FOGCOL` with `FGE = 1` is the one-boot probe that proves the whole wire path before you touch the fog math.

Fog params ride the kernel context — mind the qword under it. Generated VU1 microcode addresses its scratch through negative offsets from an XTOP-derived pointer, and the context's last qword can sit flush against the double-buffer base. A one-qword underflow then stomps that slot every frame. This is invisible for two decades when the stomped slot holds something tolerant (an unused clip constant); put your fog parameters there and it zeros them every frame — solid fog, insensitive to every parameter you change. The fix is a sacrificial pad qword between the context and the buffer base. The bisection that finds it: hardcode the fog parameters as microcode immediates. If fog then works, the math is right and the bug is delivery, not computation.

GS fog is screen-space, not perspective-correct. The interpolated F is linear in *screen* space, where a per-pixel fog unit (the Dreamcast PVR, say) fogs from true $1/W$. On a large receding surface with few vertices this diverges badly: a single center-to-rim ground fan viewed from a low camera reads as solid fog, because perspective compresses the far ground into a thin screen band and the screen-linear lerp paints mid-fog onto the near pixels. The fix is geometric — sample F where the depth actually changes. Subdivide the surface so vertices land at real depths: radial rings for a ground disc (denser across the fog band), and segments along any long wall receding in Z. Do **not** switch F to a $1/W$ metric to "fix" the interpolation — it will interpolate perfectly and then no longer match a reference curve built on eye-Z. Sony's own fog macro suite (`FogSetup` / `VertexFogLinear` / `CreateGsFog`, in the toolchain's VCL includes) is the reference for the kernel side; it is worth reading before reinventing it.

Manual reference: GS User's Manual, Drawing Function (fog, the F field, PRIM/ PRMODECONT); Image Data Transmission (giftag REGS and XYZF2 packing).

Chapter 20 - Fork Stability, Per-Vertex Alpha, and Batched Submission

Three unrelated-looking problems that all trace back to how a 2001-era fork feeds the GS: an intermittent whole-machine freeze, alpha that silently dies, and an EE that loses to a slower CPU. All three are pipeline-shape problems.

The interrupt handler that services one event freezes the machine. A GS interrupt handler written as `if (SIGNAL) ... else if (VSYNC) ...` services exactly one event per interrupt. But the GS holds its interrupt line while *any* unmasked CSR bit is set, and the EE interrupt controller latches on edges: if both events are pending at once (or a second arrives during the handler), the un-serviced bit keeps the line high, no new edge ever latches, GS interrupts stop, and the next wait-on-render-or-vsync semaphore sleeps forever. The EE is wedged; the IOP and its network loader stay alive, so the machine looks half-dead and a hardware reset combo still works — the exact signature. It is a per-frame dice roll that gets worse as frames get longer (a ~17 ms frame drifts the end-of-chain signal across vblank). Fix: drain in a loop — service *and* clear every set CSR bit, re-reading CSR until it is clean, then exit. Diagnosing this needs a deadman watchdog (below), because by the time you notice, the main thread is already asleep.

The color breadcrumb, escalated: a watchdog that names the wedge. Painting a live GS `BGCOLOR` per checkpoint scans out as rainbow margin bands — do not do that on a shipping frame. Instead *latch* a checkpoint color, and let a vblank interrupt handler (which still fires while the main thread is parked on a dead semaphore) stamp the latched color into the raster margins after the frame counter has stalled for a few seconds — and *blink* it against a second color read live from the DMA/VIF/GS status registers (DMA busy plus VU1 executing = VU1 hung; DMA idle with a signal pending = an unserviced interrupt; idle with nothing pending = the signal was lost). The checkpoint color says *where* the EE got to; the machine-state color says *why* the chain never finished. This single tool found both the interrupt-drain freeze above and a wrong-microcode VU1 wedge.

An uninitialized bitfield is a heap-layout lottery crash. A renderer registry that initializes its requirements struct field by field can leave a padding bitfield (`unused : 12`) holding heap garbage. Capability matching never sets those bits, so every match fails, and a not-found path that indexes one past the table — guarded only by an assert that compiles out in release — makes a virtual call through garbage. Whether it crashes depends on that build's allocation luck; it can hide for twenty years. Zero the whole struct before the field init, and give the release build a real no-match guard that keeps the current renderer and prints the requirement bits (those printed bits are what expose the garbage). Note the guard is a tripwire, not a fix: "keep the current renderer" on a genuine mismatch feeds the wrong microcode and can wedge VU1.

Lit renderers discard per-vertex alpha in the last instruction before the GS. A stock lit vertex kernel sets each output vertex's alpha from the *constant* material diffuse, computed once before the loop; the per-vertex color path only ever drives RGB. So anything that animates alpha through the per-vertex-color path — the usual workaround, since there is often no unlit per-vertex-color renderer — renders fully opaque, with the blend state and the alpha bytes all provably correct and zero visual effect. The alpha dies in `finish_colors`. Fix is a kernel variant that loads each vertex's own alpha (the `.w` the stock loader skips), scales it into GS 0..0x80 range, and merges it into the output color. Make it

backward-safe: an opaque stream baking alpha 255 $\rightarrow$ 1.0 $\rightarrow$ 128 must come out byte-identical, and constant-material renderers keep the stock path.

Pipeline shape beats clock speed: batch the clipper output. A 294 MHz EE lost to a 200 MHz SH4 on the same scene, because the PS2 path re-emitted every software-clipped vertex through per-vertex immediate calls (each a call chain plus several buffer appends) while the other machine submitted one batched client-array draw. The fix is the same on the EE: append clipped vertices to DMA-visible scratch arrays and flush one array draw per surface. The fork ref-DMA's client arrays straight to VU1 (float-only, tightly packed; lit paths still need a normals array — a single shared constant normal works). Two disciplines: the scratch must be double-buffered on frame parity, because the frame's DMA chain reads it asynchronously; and consecutive same-state draws merge, so order by state. The payoff beyond speed is memory — the immediate-vertex buffers shrink from a whole-frame worst case to HUD scale, refunding megabytes.

Manual reference: GS User's Manual, CRTC and the CSR/IMR interrupt registers; Drawing Function (alpha blending, the 0..0x80 alpha convention); VU User's Manual for the kernel scratch model.

Chapter 21 - Coexisting Mipmaps and the Long-Run Reset

Two topics that both surface only after the easy path works: making more than one mipmapped texture legal, and proving a long session leaks nothing.

MIPTBP is per-texture state — put it in the settings packet. A first mip implementation that writes `MIPTBP1/2` once, globally, via a raw register packet makes "exactly one mipmapped texture may exist" a load-bearing invariant — fine for a lone floor, fatal the moment a scene wants several mipped tiles. But `MIPTBP` is context state exactly like `TEX0 / TEX1 / TEXA`: its home is the texture's own settings packet, re-sent on every bind. Bake the pyramid addresses into each texture at create time and any number of mipped textures coexist; textures with no mips carry a zero `MIPTBP` the GS never dereferences. One corollary bites on a VRAM-layout switch that wipes the slot map: a fork re-uploads a texture lazily on its next *bind*, but pyramid levels are never bound (only sampled via `MIPTBP`), so after a wipe the re-uploaded base samples its levels from reassigned memory — correct up close, wrong in the distance. Every mipped texture must be deep-released before such a switch and recreated after.

Locked mip pyramids are a slot budget, not a page budget. Each mip level is its own pinned allocation from the fixed slot partition, so $N \times \text{levels}$ *locked* small slots, permanently — independent of the trivial VRAM the texels occupy. A batch of tiles with full pyramids can demand far more one-page slots than the pool holds; the locks then starve the CLUTs, which LRU-fight the leftovers and produce full-screen palette corruption while untextured and CLUT-less draws stay normal (the diagnostic tell). Before enabling mips on a batch of tiles, count $\text{tiles} \times (\text{levels} - 1)$ against the small-slot pool and recarve idle large slots if needed.

A 1-bit alpha key can be mipmapped — if you pick the filter for the blend. The blanket rule "never mip a 1-bit-alpha texture" exists because a box filter averages the key toward zero. A custom mip builder escapes it, but the correct filter depends on how the tile draws. A punch-through (replace) tile wants RGB averaged over the *covered* texels only and the key re-thresholded at $\geq$ half coverage — it conserves coverage, so a minified field of holes reads as a solid bright surface. An additive (light) tile wants a plain box average (the black gaps pull RGB down) with the key set at *any* coverage — it conserves energy, so a far field of small lights dims smoothly like real distant lights. The whole- pyramid alpha behavior is gated by that texture's `TEXA` (the base texture's bind carries it; levels are only sampled). And a related default to know: a fork's `TEXA` often ships as "identity" (`ta0 = 0x80`), which expands a 16-bit texel with alpha bit 0 to alpha 128 and passes a mid-threshold alpha test — so punch-through tiles render as solid quads until you override `TEXA` per texture (`ta0 = 0`).

The long-run reset is a memory-conservation proof. A "reset the whole game" path — unload every asset, re-init, reload — is rarely a shipping feature, but getting its per-cycle memory delta to a flat zero is the only real proof you own your allocations. Instrument it: after the post-unload drain each cycle, snapshot system RAM (`mallinfo().uordblks`) and live GS/VRAM (a fork memory query on PS2, the KOS texture-memory counter on Dreamcast). Judge the delta against a *running minimum* — the true drained floor — not against the previous cycle: a real leak raises the floor monotonically, while allocator fragmentation oscillates above a stable-or-dropping floor (the bookkeeping can even read below the steady value — not a leak). Comparing to the last cycle cries "leak" on a conserving-but-

fragmenting run. When a reset OOMs after N cycles, the instrument names the growing pool and you hunt one leak at a time.

The leak the reset exposes: your fork keeps the texels pointer. The largest such leak found here was ownership, not a missing free. A raylib-style loader allocates a DMA texels buffer per texture and hands it to the fork's `glTexImage2D`; the fork *keeps that pointer* — it re-DMA's the texels from RAM on every GS-LRU re-upload, so the buffer is not a transient staging copy and the intuition "the GS has it now, free the RAM" is wrong. But if the fork's texture never had its "free image on exit" flag set, deleting the texture frees the GS slot and never the RAM — a quarter-megabyte per texture, a megabyte per reset cycle, invisible to the loader's own unload path because the leak lives inside the fork's ownership model. The fix is an explicit ownership transfer (a `take-ownership` call after `glTexImage2D`, backed by a "free on exit" setter on the texture object), applied narrowly so paths that manage their own buffers stay untouched. The general rule: any allocator that hands the fork a buffer and then forgets it is leaking — the fork frees the image only if told it owns it. Two smaller siblings round out the set: model/material textures need a *deep* unload (dedup shared ids) rather than the desktop assumption that unloading a model frees its skins, and stage-independent install helpers that rebuild a texture every call must bake it once (`if (already_built) return;`) or they leak one per reset.

Manual reference: GS User's Manual, Local Memory (page geometry, TBP/MIPTBP addressing) and Drawing Function (TEXA, the alpha test).

Chapter 22 - Writing VU1 Microcode Through VCL: The Clip Renderer

Relevant manuals: VU User's Manual Ch 3 (flags, hazards, pipelines) and the VU Instruction Manual for per-instruction flag/latency tables; GS User's Manual Ch 3 for the packed GIF vertex formats the renderer emits.

The endgame of the clipping story (Chapter 9): the EE software clipper moved into a custom ps2gl VU1 renderer that performs full five-plane Sutherland-Hodgman — near plane plus four guard-band side planes — directly in microcode. The EE submits raw eye-space triangles; VU1 classifies, clips, interpolates STQ, and emits GS packets. Same pixels, and the EE sheds the entire clip-plus-submission cost (a city pass went from 11-20 ms to ~9-14.7 ms at higher vertex loads, with the EE's draw-call time at ~4-5 ms/frame). This chapter is what that cost to learn: the extension seam is clean, but the VCL toolchain has real bugs, and a naive port of correct C onto correct VU semantics still fails until you know them.

The extension seam needs no changes to ps2gl's renderer selection: `pglRegisterRenderer` plus `pglRegisterCustomPrimType` (a GLenum with bit 31 set), keyed to a user-properties bit with a requirements mask of `~0xfffffffffull` so matching ignores all standard state. Three traps: the geometry block fills per-prim metadata only for standard prims (set `NumVertsPerPrim` and friends yourself in `DrawLinearArrays`); the giftag builder cannot map a custom enum (pass the equivalent standard prim down); and a failed match silently keeps the *current* renderer, which renders identically if your microcode started as a clone — prove selection with the renderer NAME, not pixels.

Variable-output geometry (one triangle in, up to six out) breaks the stock 1:1 pipeline assumptions. The pattern that works: store each output triangle optimistically at the output cursor and commit (advance cursor and a vertex counter) only if kept; after the loop, rebuild the giftag from its template with the real count and store it over the packet head before `xgkick`. A buffer whose triangles all drop kicks a bare `NLOOP=0 EOP=1` giftag — legal and hardware-verified. Stock lighting passes walk output 1:1 with input and cannot survive 1:N; resolve color in the main loop.

The VCL toolchain bugs, in order of pain:

1. **vcl does not model VU-memory aliasing between pointer registers.** It will hoist one pipeline stage's loads above the previous stage's stores to the same address through a different pointer — the consumer reads the previous triangle's data, and which qwords hoist is a scheduler roll of the dice per regeneration (positions can survive while texture coordinates die). Fence every store-to-load seam with a branch to the next line: the label is a basic-block boundary vcl cannot schedule across, and its late peephole erases the branch so the fence is free — and invisible, so verify by store/load order in the disassembly, not by looking for the label.
2. **vcl "regenerates" MAC flags with ABS, which sets no flags.** Never use `fmand` for a new sign test; when the scheduler separates producer and consumer it inserts a flag-inert ABS and the test silently reads stale flags. Extract signs deterministically: clamp a copy to ± 2047 , `ftoi4`, `mtir`, mask bit 15.
3. **Pipeline Q with multiple consumers is schedule-fragile.** Two `mulq`s off one `div` are only correct while the scheduler keeps both inside that div's window — the next regeneration may not.

Land Q in a register at the divide (`addq.x qreg, vf00, q` — `vf00.x` is zero) and broadcast-multiply from the register. Note `vcl` rejects an explicit `waitq` outside RAW mode; the single-consumer `addq` is the whole fix.

Two hardware traps that cost hardware round-trips: `vf00.w` is ONE, so building a `.w` constant additively on `vf00` silently adds one (the near plane became `1+near`); and `ps2gl`'s buffer splitter can hand a triangle-list microcode `3n-1` vertices (an odd-split adjustment from the strips era), which wedges any vertex loop that terminates on pointer equality — `VU1` spins forever and the console hard-hangs. Round EE-side per-buffer caps to a multiple of six AND make the loop overshoot-proof (terminate on "remaining less than one triangle").

Validation doctrine for microcode (extends Chapter 17): a standalone test ELF with a build tag banner; textured validation via a procedural checkerboard (texcoord bugs are invisible untextured); geometry that targets the mechanism (tilted discs straddle the near plane, flat discs pop whole for the `NLOOP=0` case, a static marker separates "draws nothing" from "culled everything"); every per-buffer cap exercised by a draw that spills it; and the new path run with `ps2gl` clipping disabled so its own protection carries the load. Know the reference column's limits: stock `ps2gl` silently DROPS guard-band-busting triangles, so the stock side of an A/B can be blank exactly where the interesting cases are. And two GS-level phantoms that mimic clipper bugs: overlapping coplanar geometry from two pipelines z-fights into a static horizontal comb (the two paths round depth differently), and a single giant triangle with huge UV magnitude streaks on the GS's reduced-precision STQ interpolation no matter who clipped it — test with game-scale tiles.

Manual reference: VU User's Manual Ch 3.3.5 (instruction flag table — ABS sets none) and Ch 3.4 (hazards: Q/ACC/I have no interlocks; waitq stalls both pipes).

The output cap is an input contract

A clipping microcode's output buffer is finite, and ours guards it the honest way: when a clipped fan will not fit under the per-buffer cap (`kCCapVerts = 39`), the fan is dropped and the buffer's remaining triangles with it — better a missing triangle than a compound packet overrunning into the next buffer half's header and wedging the GIF. That policy is sound *for the workload the cap was budgeted against*: wall quads arrive mostly inside (`8 tris = 24 verts`), and a clipped one adds two or three.

It is structurally unsound for giant polygons, and it fails silently. A screen-covering triangle straddles three or four frustum planes at once, clips to a 5-7 vertex polygon, and fans back out as 9-15 vertices; a few of those sharing one 8-triangle input buffer sail past the cap, and whole fans vanish. The symptom on hardware is huge missing triangles that come and go with camera position — whichever triangles happen to share a buffer decide the frame. In HyperSolar this surfaced as vanishing ground over Central Park, the one place the camera flies inside giant land-platform polygons with nothing drawn over them.

Three durable conclusions. First, the cap is part of the renderer's interface: a stream whose primitives can individually fan past the remaining budget must not ride that renderer, however rare the overflow — "rare" is exactly what "missing at times" looks like. Second, big-polygon streams belong on a pre-clipping path (EE Sutherland-Hodgman): every batched triangle then arrives fully inside, expansion is zero by construction, and such streams are tiny anyway (the land platform is 24-156 vertices). Third, a silent-by-design drop deserves a counter — two guards whose only observable output is a missing wedge cost a hardware session to localize; a dropped-fan count in the log names the failure in one run. A

practical forensic shortcut fell out too: runtime mode toggles that reroute the *same* geometry through different renderers (our SELECT X2/GS/EE cycle) convict the microcode with zero builds — holes in X2 that vanish in GS/EE cannot be the geometry builder's fault.

Refinement: preserve the input batch and spill output inside VU1

The Central Park fix made a reasonable but incomplete generalization: giant polygons belong on the EE pre-clipper, while ordinary wall quads satisfy X2's 39-vertex output budget. A later frozen camera A/B disproved the second half. One fat building wall lost a triangle in X2 and X2D but not in GS or EE mode. Even small input primitives can expand into an unsafe *batch* when near and side planes intersect them together.

Splitting correctly before DMA repaired the triangle and regressed the frame. Conservative and exact-count splitters fragmented the normal four-descriptor VU activation toward one descriptor: the wide path rendered about 10,029 verts at sync 1.74 ms / city 7.95 ms / 60.0 fps, while a safety splitter rendered fewer verts yet reached sync 4.30-4.58 ms and 56-57 fps. Other variants reached 6.22 ms sync / 48.5 fps. The lesson from Chapter 23 applies inside the clipper too: submission shape can dominate arithmetic and vertex count.

The lossless solution leaves the wide input batch alone and spills variable output between two VU-memory arenas during one microprogram invocation. The x2 buffer half has 292 free qwords. A compound arena needs two GIF tags; each emitted vertex consumes three wall qwords and three window qwords. Two arenas therefore obey:

$$4 + 6 * (A + B) \leq 292$$

A=30 and B=18 use the region exactly. A triangle clipped by five convex planes becomes at most an eight-vertex polygon (3 + 5) and a six-triangle fan, or 18 output vertices, so the smaller arena always fits one whole fan. When the current arena lacks room, VU1 closes and `KGICK` s it, continues in the other arena, and alternates after the next kick. Alternation is the ownership rule: do not overwrite a packet PATH1 may still be consuming. The ordinary eight-triangle/24-vertex batch fits A and keeps its original one-activation, one-compound-kick path.

Do not hold arena address/capacity in integer registers across a five-plane S-H clipper; VCL ran out of VI registers. Reconstruct the current tag only at a spill point from `next_output - 3*out_count - 1`. After regeneration, verify generated store/load and kick order, the exact-image hash, decoder relocation, and combined micro-memory budget. The hardware-green image is X2 810 instructions plus the unchanged 56-instruction X2D decoder at PC810, tail-calling X2 PC6 (866/2048 total).

Hardware validation repaired the frozen wall angle and held every steady-state 10-second window at 60.0 fps through a 190-second idle traversal. The dense windows remained at sync 1.73-1.74 ms with roughly ten thousand vertices per frame. Keep giant land on its already-proven EE route: a new capacity mechanism is not permission to broaden a renderer lane without a measured reason.

Chapter 23 - Measuring the Wall, and Kicking Twice

Relevant manuals: GS User's Manual Ch 3 (drawing function, per-primitive `ABE`, `ALPHA_1`) and Ch 4 (image data transmission — the GIF packet formats each kick emits); EE User's Manual (DMAC/VIF/GIF paths) for what a "flush" actually costs.

Chapter 22 moved clipping off the EE and into VU1 microcode. The city got faster, but not fast enough, and the natural next move — offload *more* — turned out to be the wrong instinct. What the frame needed first was not another optimization but a measurement precise enough to say which side of the pipeline was starving.

Two counters, one question

A per-pass timer answers *which* pass is expensive. It cannot answer *why*: a pass that spends its milliseconds building DMA chains, stalling on VIF1, or waiting for VU1 looks exactly like a pass that spends them in EE arithmetic. The distinction matters enormously, because the two have opposite cures.

The fix is a second counter wrapped around the single choke point every batched draw exits through — `glDrawArrays`. Accumulate the time spent inside it, plus the draw and vertex counts, and report both counters together. Now the frame splits: time inside `glDrawArrays` is downstream (chain building, VIF1 bandwidth, VU1, GS); the pass total minus that is the EE's own vertex loops — transform, clip, the cache-cold walk over the geometry stream, and the write-combining copy into DMA scratch.

For HyperSolar's city, mid-flight, the answer was roughly four milliseconds inside `glDrawArrays` against eight in the EE loops, at about ten thousand vertices. That is on the order of **280 EE cycles per vertex** — an order of magnitude more than the arithmetic in those loops can account for. The wall was never clip math. It was memory traffic: reading vertices that had fallen out of cache, and writing them again into uncached DMA memory.

Two consequences followed immediately, and both were unwelcome.

First, a chunked display-list design — compile static slabs of the city once, call them per frame, let VU1 do everything — was not merely difficult on this fork; it was pointless. It removed arithmetic and left the per-vertex read and copy exactly where they were. (It was also structurally impossible: ps2gl's list manager is a bump allocator that wraps at four thousand IDs and asserts on collision, each list lazily claims four 256 KB DMA buffers, frees defer a frame, and colors and texture binds bake at compile time. Display lists are for install-once rigid models. A streaming, tinted, alpha-animated world is not that.)

Second, a "raw submit" fast path — skip the clipper, copy vertices straight into scratch — was measured and moved nothing, for the same reason. Both experiments were built, measured, and reverted. The counter had predicted both outcomes.

What the counter *did* endorse was the opposite move: touch fewer vertices.

One transform, two kicks

The city drew each building wall twice. The first pass laid down an opaque wall, its texture modulated by a per-building tint. The second pass drew the lit windows over it, additively, from the same vertices with a different texture. Identical geometry, submitted, transformed, and clipped twice.

The Graphics Synthesizer offers no way to avoid the second *rasterization*: it has no multitexturing, and no `TFX` mode tints a wall texel while sparing a window texel beside it — which is precisely why the window pass exists at all, and why baking both layers into one texture (the obvious "just Photoshop it" idea) cannot reproduce the look. The second rasterization is irreducible.

The second *transform* is not. A companion VU1 renderer clips and transforms each shared vertex once, then issues two `XGKICK`s from the same transformed buffer: the opaque wall primitive with `ABE=0` and the per-vertex tint, then the window primitive with `ABE=1`, a constant color, and the window texture. Measured on hardware, the city pass fell from 11.74 ms to 8.33 ms under load — a 29 percent reduction, from deleting work that was never visible to the GS.

Four details cost a debug cycle each, and generalize to any multi-kick renderer:

The per-kick texture switch is an **A+D settings block**, not a bind. Copy the texture's own settings block — its GIF tag plus `TEXFLUSH`, `CLAMP`, `TEX1`, `TEX0`, `TEXA` and both `MIPTBP` registers — wholesale, so punch-through `TEXA` keying and the custom mip pyramids of Chapter 21 ride along verbatim, and grow the `NLOOP` to append the blend register the kick needs.

Constant colors enter at **x128, not x255**. A textured primitive's modulate identity is 128, and the fork's own `GetMaxColorValue` says so; scaling by 255 renders at double brightness. This is the same trap that made untextured translucent geometry refuse to fade in Chapter 20, wearing a different hat.

A sentinel disables the second kick: writing `-1.0` into the window color slot turns the renderer into a wall-only path, so one renderer serves both the combined draw and any single-textured per-vertex-colored draw. Cheaper than two renderers.

The flush must be **synchronous**. `ps2gl` defers geometry to the next state change, while this renderer captures both textures at flush time. A deferred flush therefore executes with the *next* draw's bindings, and the city flickers building by building. Flushing immediately pins the capture to the state that produced it.

And afterwards, dirty `ps2gl`'s blend cache: the window kicks program `ALPHA_1` behind the library's back, so the next blended draw must resend its own function or inherit an additive one.

What a clipper was quietly doing for you

Moving clipping to VU1 has a consequence that no one writes down. A software clipper **culls as a side effect**: a triangle wholly outside the frustum clips to zero vertices and is silently never emitted. Take the clipper out of the EE and that free rejection leaves with it. Every triangle in the buffer is now transformed, written to scratch, DMA'd across VIF1, and processed by VU1 — only to be discarded there.

The counters showed it plainly. After the double-kick landed, the VU1 path was submitting about ten thousand vertices per frame while the old two-pass EE path submitted about ten and a half thousand — nearly identical, when the new path should have been five thousand *lower*, because it no longer resubmits the wall stream. The missing five thousand were off-screen triangles that the EE clipper had been rejecting all along. The geometry streams span an entire build window; the camera sees a cone.

The remedy is cheap and belongs on the EE, because the eye coordinates it needs have already been computed by the transform: build a five-plane outcode per vertex, AND the three outcodes of a triangle, and drop the triangle when the result is non-zero — all three vertices outside the same plane means it can never be visible. One subtlety decides correctness: a vertex behind the near plane must contribute **only** the near bit. Its lateral position is meaningless once $z \leq 0$, and emitting side bits there will reject a triangle that straddles the near plane and is partly on screen.

Clipping is VU1's job. Deciding whether geometry is worth sending is still the EE's.

Discipline for the counters themselves

Report per-frame averages over a **time** window, not a frame-count window. At variable frame rates a fixed frame count covers a different stretch of the world each time, and a single stage-load stall will dominate a short average. Ten seconds is a comfortable window.

Tag every line with the configuration it describes, and **restart the window when the configuration changes**. An A/B toggle that leaves the accumulator running silently blends two configurations into one average and produces a confident, meaningless number.

Finally: governor and LOD bands are calibrated against a *cost curve*, not against geometry. When the renderer gets faster the bands do not know it. After both upgrades in this chapter the city's autotune still sat pinned at its throttling floor, holding back detail the frame could now afford. Every time the pipeline changes, the tuning that was measured against the old pipeline becomes a lie.

Chapter 24 - The Heap Smasher: Forensics When the Crash Site Lies

Relevant manuals: EE Core User's Manual Ch 4 (exception handling — reading `Cause`, `BadVAddr`, `EPC`); the newlib allocator source (`_mallocr.c`) for what a chunk header looks like on the ground.

Chapter 23's double-kick renderer shipped with a bug that no amount of frame measurement could see: a one-line buffer under-declaration that corrupted the heap on every wall-only draw and crashed the game minutes later, in a different subsystem, on a button press. This chapter is about how such a bug presents, why the obvious readings of the evidence were wrong twice, and the small toolkit that finally cornered it in three hardware runs.

The signature: an exception inside the allocator

The console showed an EE TLB load exception with `EPC` resolving (`addr2line`, always `addr2line` first) into `__malloc_update_mallinfo`, called from `_mallinfo_r`, with `BadVAddr 0x00000004`. Read that address the way the machine does: some chunk's `fd / bk` link was `NULL`, and the walker dereferenced `NULL + 4`. The allocator was walking its free list and stepped on a chunk header full of zeros.

This is the single most important fact about the whole class: **the crash site is the first reader of the damage, never the writer**. The heap was corrupted earlier — possibly minutes earlier — by code that has long since returned. Debugging `mallinfo` is debugging the coroner. The crash moving between builds, or appearing "when I press START" (the debug HUD's RAM row called `mallinfo()`), is the same misdirection: the button did not break anything; it was merely the next reader.

The tool: the walker itself, turned into a validator

The function that crashes on a corrupt heap is, by that very property, a perfect detector for one. `mallinfo()` walks every chunk and faults on the first bad link — so call it on purpose, everywhere, with one discipline: **print a label before the walk, and a second line after**. On a healthy heap you get `chk x / ok x` pairs; on a corrupt one the output ends at a `chk` whose `ok` never came, and that label names the phase. (The first version of this instrument printed only after the walk and was therefore silent on exactly the runs that mattered — an instrument must be proven able to fire before its silence means anything.)

Deployed as a ladder — first around the stage-load phases, then around the frame's draw passes, then around the branches inside the guilty function — it took three hardware runs to descend from "the stage-3 load kills it somewhere" to one call: the first wall-only draw through the x2 renderer, 24 vertices of land platform.

The false exit: padding that "fixed" it

Mid-hunt, a 16-byte canary added past one large allocation made the crash unreproducible, and the tempting reading was Heisenbug — layout-sensitive, file it away. The correct reading is a classification: a *relative* overrun (code running off the end of its own buffer) moves together with its victim when the heap slides, so padding cannot hide it; only a write to an effectively **absolute** address — a stale pointer,

a mis-built DMA tag, or a fixed-size overrun inside one specific heap object — goes quiet when everything shifts 16 bytes. The suppression was not noise. It was the bug stating its addressing class, and it pointed directly away from the padded buffer and toward some other heap object with a fixed layout.

The writer: a staging array sized from a comment

The x2 renderer stages a 23-quadword prefix block per buffer — window color, two 10-quadword texture-settings blocks (giftag + nine A+D registers each), and a prim giftag template — unpacked to VU memory with `packet.Add(Pfx, 23)`. The C++ member backing it was declared `uint128_t Pfx[21]`: sized from a header comment that described the settings blocks as 9 quadwords. Every draw wrote `Pfx[21]` and `Pfx[22]`; the wall-only branch's `memset(&Pfx[12], 0, 11 * 16)` wrote 32 bytes of zeros past the object. And the object lives on the heap (`new CClipTriX2Renderer`), so those zeros landed on the next malloc chunk's header — the NULL `bk` the walker found. The fix was one character. The lesson is structural: when a size is asserted in three places — the array, the transfer call, and the microcode's reserved range — derive all three from one constant, and if you must pick one to trust, pick the transfer call you can grep, never the prose.

Rules this chapter adds

An exception inside `malloc / free / mallinfo` means the heap died earlier, elsewhere; resolve `EPC and ra`, then hunt the writer, not the reader. `mallinfo()` is a drop-anywhere heap validator; label before the walk. A perturbation that suppresses a memory bug is a localization result. Custom renderers are heap objects — a member-array overrun is heap corruption. And scaffolding is scaffolding: the per-frame walk costs real EE time and leaves with the fix.

Chapter 25 - The Sixty Lock: One Kick, One Page, and the Flat Matrix

Relevant manuals: GS User's Manual pp. 47-49 (the two register contexts and `PRIM.CTXT`), Ch 8.3/8.5 (block arrangement tables — the source of the packed-mip offset magic), and the trilinear/LOD sections the mip fix leans on.

Chapter 23 ended with the double-kick renderer: transform the shared wall geometry once, kick the GS twice. This chapter is how the city went from "dips to 51 under load" to a combat-verified, locked 60 fps in 448p — and it is really two stories: an architecture the GS manual had been advertising all along (its second register context), and a texture-layout pathology that survived four hardware experiments precisely because none of them touched the one thing that mattered.

One compound kick: the second context is not decoration

The double-kick still paid a hidden tax: per-micro-buffer settings prefixes, each with a `TEXFLUSH` — about 206 per frame at dense load. `TEXFLUSH` stalls the GS until in-flight drawing completes, then invalidates the texture cache, so the profile `READ` as a fill wall while much of it was serialization bubbles. **A stall pattern scales with draw count; true fill scales with coverage** — count texture-path state events per frame before blaming pixels.

The fix is the GS's own design: two persistent register contexts exist exactly so intermixed primitives can switch state with a bit in `PRIM` instead of repeated register loads. Walls draw on context 1 (live GL state, `ATE` pinned off), additive windows on context 2 (window texture, `ALPHA_2`, `TEST_2` — programmed once per pass), inside `ONE XGKICK`: wall gifttag `EOP=0`, window gifttag `EOP=1` closes the packet, and the zero-vertex case still emits the empty window tag so the kick terminates (hardware-proven over a 2700-frame torture rotor). Worth ~2.1 ms of GS wait at matched wall density. Two traps are load-bearing: **context 2 is not yours alone** — `glClear` draws through a context-2 draw env every frame and stomps it, so the renderer re-arms per pass; and the wall pass rides `LIVE` context-1 state, so whatever the old prefixes used to force (blend, alpha test) must now be pinned explicitly.

The land platform: four flat experiments and the page walk

The remaining dips tracked the city's ground. A debug layer kill-switch priced it instantly — land off, 60 fps, every time — and then four serial hardware experiments tried to keep the layer visible but cheaper: draw it after the city so hidden pixels Z-fail; make the roads write depth so the street corridor rejects the land beneath; reshape the primitives (32u slivers, 128u strips, giant fans); finally draw it with the depth unit fully off (`ZMSK=1 + ZTST=ALWAYS` — no Z read, no Z write). **All four were flat.** That flatness is the finding: depth outcome, draw order, primitive shape, and the whole Z unit eliminated, the cost follows exactly one variable — textured fragments rasterized. When every state-side experiment is flat, the cost is on the sampling side.

The sampling-side suspect: the land texture's mip levels each lived in their own GS page, and a grazing-angle ground plane keeps every pixel `BETWEEN` two LOD levels, so trilinear fetches two page-separated texels per pixel — a per-pixel page walk worth ~1.4-2.0 ms, roughly ten times the raw fill cost of the same pixels. A one-letter CSV test (mips off) jumped every immovable window to the ceiling — but note

the audit correction: that removes the page reloads AND trilinear's inherent cost together, so it only localizes the path. The proof was the fix itself: pack the whole 64x64 pyramid into ONE page (identical texels, identical filtering, only the layout changed) and the windows locked at 60 with mips on. Scatter was the cost; trilinear is effectively free once both levels share a page.

The packed path has its own contract: magic block-offset tables from the GS manual's block arrangement tables (PSMT8 packs L0-L3 in one page; PSMCT16 packs L1-L3 beside a full L0 page), MIPTBP TBW=1 for the sub-64 levels while the upload keeps DBW=2, one locked memory-area owner per pack released exactly once, no lazy re-allocation allowed to retarget TEXO away from the pack — and scope discipline: only the exact tested pyramid shape takes the packed path; everything else stays on the proven legacy allocator.

What did NOT work, so nobody rebuilds it

Front-to-back submission (no early-Z to feed — sorting cost with zero GS gain, measured worse than build order). Draw-order/Z-fail tricks on a fetch-bound surface (Z-fail saves only the write RMW, and that slice measured ~zero here). Strip-shape tuning (the column-count telemetry correlated beautifully with the dips and was a confound — the A/B is the judge, never the correlation). And a process rule that made the whole hunt converge in seven hardware slots over two days: one change per slot, expected log signature stated before the run, matched route, and any visual regression rejects the slot even when the numbers are flat.

Rules this chapter adds

- Two materials intermixed = two GS contexts and one compound kick, not two kicks and not state resends. Context 2 must be re-armed: glClear owns it once a frame.
- Count TEXFLUSHes before blaming fill. Stalls scale with draws; fill with coverage.
- Keep every mip pyramid page-local. Scattered levels turn trilinear into a per-pixel page walk on any large grazing surface; pack the pyramid.
- The flat matrix localizes invisible GS costs: vary depth outcome, order, shape, and the Z unit one slot at a time — all flat means sampling-side.
- A layer kill-switch prices a subsystem in one toggle; judge by the GS-wait delta at matched windows. It is an upper bound: partial tricks may recover none of it.
- Front-to-back is worthless on the GS. Submit less or sample cheaper; never sort.
- One change per hardware slot, signature declared before the run, and a visual regression rejects the slot regardless of the numbers.

Appendix A - Quick Rules

- Standalone boot: reset IOP, wait sync, re-init SIF RPC, apply SBV patches, then load buffer IRX modules.
- ps2link success is not standalone success.
- Use GS color breadcrumbs when `printf` is unavailable.
- Keep the PS2 backend as a backend library. No scene ownership, no direct game state reads, no hardcoded asset filenames.
- Avoid libgraph on top of ps2gl.
- Use 448p, not 480p, for the real progressive layout.
- Align framebuffer height to GS page rows.
- Pair 16-bit color with 16-bit Z; pair 32-bit color with 24-bit Z.
- DTV progressive needs the correct display magnification.
- Re-zero GS BG_COLOR every frame and before long mid-frame stalls.
- A crash inside `malloc` / `mallinfo` names the victim, not the writer — the heap died earlier, elsewhere. `mallinfo()` itself is a drop-anywhere heap validator; print the label BEFORE the walk.
- Size VU1 staging arrays from the transfer count, never from a comment.
- Padding an allocation and the bug vanishing is a localization result (absolute- address write), not a Heisenbug.
- Batch by state. State changes beat vertex count as the usual cost center.
- Keep one uniform color per batch, or bucket by quantized color.
- Never emit empty `rlBegin` / `rlEnd` batches.
- Treat the slot after `EndDrawing()` as off-limits for raylib/GS calls.
- Do not use `%f` in hot PS2 HUD/debug formatting.
- For always-on-top overlays, use `glDepthFunc(GL_ALWAYS)`, not only `glDisable(GL_DEPTH_TEST)`.
- Alpha-blended PS2 geometry that must fade should be textured, even if the texture is a 1x1 white pixel.
- Custom world-space geometry goes through the software clipper or a proven display-list path, not raw world-space `glBegin`.
- PGL clipping on for raw display lists unless a bounding-sphere test proves a safe bypass; PGL clipping off for software-clipped meshes.
- Keep UV magnitudes small on GS hardware.
- Use PSMT8 for most art; reserve CLUT slots and align blobs to 16 bytes.
- Put `MIPTBP` in each texture's settings packet, not a global write, and any number of mipmapped textures coexist. Deep-release mipped textures before a VRAM-layout switch (levels are sampled, never bound, so they do not self-heal).
- Enable 16-bit framebuffer dither through the ps2gl drawenv object.
- Use `mallinfo()` for EE memory and a forked ps2stuff query for live GS memory.
- For GS memory, track largest free slot, not only total free pages.
- Intermixed materials: two GS contexts, one compound kick. Re-arm context 2 — `glClear` stomps it every frame.
- Count TEXFLUSHes before blaming fill (stalls scale with draws, fill with coverage).
- Keep mip pyramids page-local; scattered levels make trilinear a per-pixel page walk.
- When depth, order, shape, and the Z unit all measure flat, the cost is sampling-side.
- Front-to-back submission is worthless on the GS — no early-Z to feed.

- One change per hardware slot; a visual regression rejects it regardless of numbers.
- Runtime raster offset wants a narrow DISPLAY write, not a full display-state resend.
- If exposing dither through ps2gl, route it through the drawenv object.
- Multiple PSMT8 textures need per-texture CLUT ownership.
- Manual GS mips work; keep MIPTBP per-texture and count locked mip slots (`tiles × (levels - 1)`) against the small-slot pool before enabling them.
- The GS fog unit is real and free: set `PRIM.FGE` , `PRMODECONT.AC` , `FOGCOL` , and a per-vertex F in the kernel; mask the ADC word to bit 15 before OR-ing F.
- GS fog interpolates in screen space — subdivide receding surfaces (rings, segments) so F is sampled at real depths; do not switch F to a 1/W metric.
- Drain the GS interrupt handler in a loop (service and clear every set CSR bit) or a coincident SIGNAL+VSYNC eventually freezes the EE.
- Zero a capability/requirements struct before field-init; an uninitialized padding bitfield is a heap-layout-lottery crash.
- Lit kernels discard per-vertex alpha (they use the material's) — animate alpha through a kernel variant that loads the vertex `.w` , not through blend state.
- Batch software-clipped output into DMA-visible arrays (double-buffered on frame parity) and flush one array draw; per-vertex immediate calls lose to a slower CPU.
- A 1-bit alpha key can be mipmapped if the filter matches the blend: coverage- conserving for punch-through, energy-conserving (plain box) for additive.
- Override `TEXA` per texture (`ta0 = 0`) for punch-through 16-bit tiles; the fork's "identity" default (`ta0 = 0x80`) renders alpha-0 texels opaque.
- A fork keeps the RAM texels pointer you pass to `glTexImage2D` and re-DMA's it — transfer ownership explicitly, or deleting the texture leaks the buffer.
- Prove a reload cycle leaks nothing by comparing RAM/VRAM against the running minimum (drained floor), never the previous cycle (fragmentation is not a leak).
- Fence every VU-mem store-to-load seam in VCL code (branch to next line) — vcl does not model aliasing between pointer registers and hoists loads over stores.
- Never `fmand` a new sign test under vcl (it "regenerates" flags with flag-inert ABS); clamp a copy, `ftoi4` , `mtir` , mask bit 15.
- Land pipeline Q in a register at the divide when it has more than one consumer; a cycle-counted `mulq` pairing dies on the next regeneration.
- `vfoo.w` is ONE — never the additive base for a `.w` constant.
- Terminate VU1 vertex loops overshoot-proof, never on pointer equality; ps2gl's splitter can deliver 3n-1 verts to a triangle microcode.
- Variable-output microcode: optimistic store + conditional commit + post-loop gifttag NLOOP patch; NLOOP=0 EOP=1 is a legal kick.
- Prove custom-renderer selection by name (`pglGetCurRendererName`), not pixels.
- Load both `libs.d.irx` and `ps2snd.irx` for EE-side `sceSd*` .
- `sceSdVoiceTrans` source is an IOP address; SPU destination is the byte address value.
- Encode SPU2 samples at 48 kHz; `PITCH 0x1000` is unity.
- Loops use loop flags; one-shots use END only.
- Music streaming belongs on the IOP in a custom IRX, not the EE render thread.
- `mass0:` is BDM/iomanX, not legacy `mass:` .
- Read IOP-DMA'd EE memory through the uncached mirror.
- Burned data discs are illegal media on stock/FMCB consoles.

- Measure before offloading: time the passes, then time inside `glDrawArrays`. The gap between the two is the EE's per-vertex loops.
- Vertices touched per frame is the currency — not triangles, not math per vertex.
- The GS has no multitexturing. Two textures means two kicks; it does not mean two transforms.
- Constant colors entering a textured primitive scale by 128, not 255.
- Moving clipping to VU1 removes the EE's free culling. Put a whole-triangle outcode reject back, and give behind-near vertices only the near bit.
- Cull small quads whole (particles, glows, billboards); never clip them. A radius cull is not a visibility cull.
- Display lists are install-once. They cannot back a streaming or tinted world.
- A crash address is not a diagnosis: resolve `EPC` and `ra` with `addr2line`. A small `BadVAddr` means a NULL base plus an offset.
- Report performance averages over a time window, tagged with the configuration, and reset the window when the configuration changes.

Appendix B - Case Study: HyperSolar PS2 Port in One Page

The PS2 target started as a skeleton platform branch: a PS2 source set, a backend translation unit, no-op audio, placeholder rendering, and enough input to cycle states end-to-end.

Milestone 1 brought up the Ninja Dynamics splash using embedded PNG data, GS-alpha-correct texture upload, and the eye-space raw GL perspective-quad path.

Milestones 2 and 3 folded the PS2 PoC environment into shared title rendering: skybox, floor, sun, haze, speedlines, lens flare, logo display list, shared font atlas, and PS2-safe texture/filter handling.

Enemy and player models use display lists with cached AABB data, explicit texture lifetime handling, and PS2-specific clipping/culling state.

HUD performance moved from per-glyph/per-rect reconfiguration toward grouped font and rect batches, proving state changes were the dominant cost.

Texture work evolved from oversized RGBA/16-bit experiments to a full PSMT8 CSV pipeline with per-asset settings, CLUT ownership fixes, 16-byte blob alignment, manual mips for floors, dithered quantization, and stage-resident loading.

Progressive video became a real 640x448p GS layout switch with 16-bit color/Z, drawenv dithering, shared screen dimensions, and a black-bracketed transition.

Audio came up in layers: pitch-clean SPU2 loops, shared mix math, one-shot voice pool, custom ADPCM encoder, then music streaming through a custom IOP IRX and BDM `mass0` storage.

Saves reached PS2 memory cards with screen settings, aligned libmc buffers, ps2link fileio barrier, BIOS browser icons, and a `glb2icn` converter.

A dense-city stage then pushed the fork past its comfort zone and produced the Chapter 19-21 material in one sustained run. GS hardware fog was wired into the vertex kernel and debugged from a red-`FOGCOL` probe through a context-stomp pad fix; the ground was re-tessellated into radial rings so screen-space fog samples at real depth. An intermittent whole-machine freeze was traced to a single-event GS interrupt handler and fixed with a drain loop, using a latching BGOLOR watchdog that names the wedge. Mipmaps moved from a single global owner to per-texture `MIPTBP` packets so the city's tiles could all mip at once, which in turn exposed the locked-mip slot budget. Per-vertex alpha, silently discarded by the lit kernel, got its own kernel variant so building fades could work at all. Finally a debug "reset the whole game" path became a memory-conservation proof: a per-cycle RAM/VRAM instrument judged against the drained floor drove the leak count to zero, the largest leak being a fork that kept — but never freed — the DMA texels pointer handed to `glTexImage2D`.

The clipper's own endgame followed (Chapter 22), and then the lesson that outranks it (Chapter 23): before offloading more work, measure which side of the pipeline is starving. A counter around `glDrawArrays` proved the city was bound by per-vertex memory traffic on the EE, not by clip arithmetic — which killed two plausible optimizations before they were finished, and pointed at the one that worked: draw the shared wall geometry once and kick the GS twice.

The final rule of the port is simple: the shared game owns meaning, timing, assets, and scene order; the PS2 backend owns the hardware-safe way to draw, store, stream, and persist those decisions.